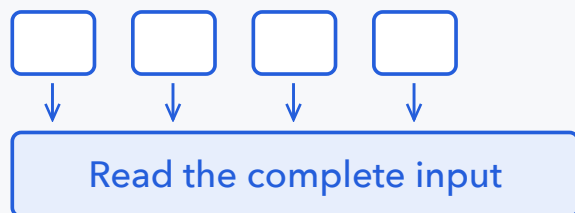


From Attention to Applications

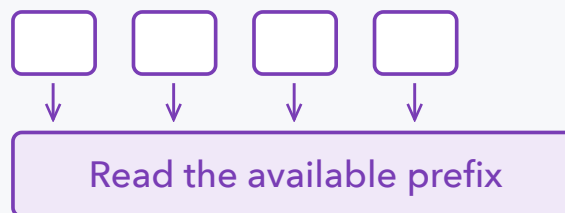
Encoder, decoder-only and encoder-decoder Transformers

ENCODER



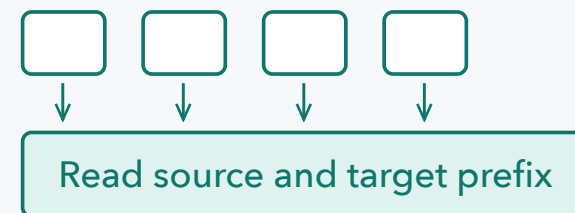
Token and sentence labels

DECODER-ONLY



Next-token generation

ENCODER-DECODER



Conditional generation

Recap: the Transformer components we have studied

1 · Positional encoding

token embedding + position

$$\mathbf{e}_i + \mathbf{p}_i$$

Token identity and order

2 · Self-attention

Q, K, V from embeddings

scores $\rightarrow$ softmax $\rightarrow m_i$

A weighted sum of values

3 · Multiple heads

head 1

head 2

head 3

Concatenate messages

Project with $W_O \rightarrow \Delta \mathbf{e}_i$

Residual update: $\mathbf{e}_i \leftarrow \mathbf{e}_i + \Delta \mathbf{e}_i$

Then the MLP and its residual update

Result: one contextualised embedding at every token position

Different tasks for the same input

Raghav goes to school in Delhi.

| What should the Transformer output?

Different tasks for the same input

Raghav goes to school in Delhi.



What comes next?

A continuation

Different tasks for the same input

Raghav goes to school in Delhi.



What comes next?

A continuation

What is each token?

PERSON ... PLACE

Different tasks for the same input

Raghav goes to school in Delhi.



What comes next?

A continuation

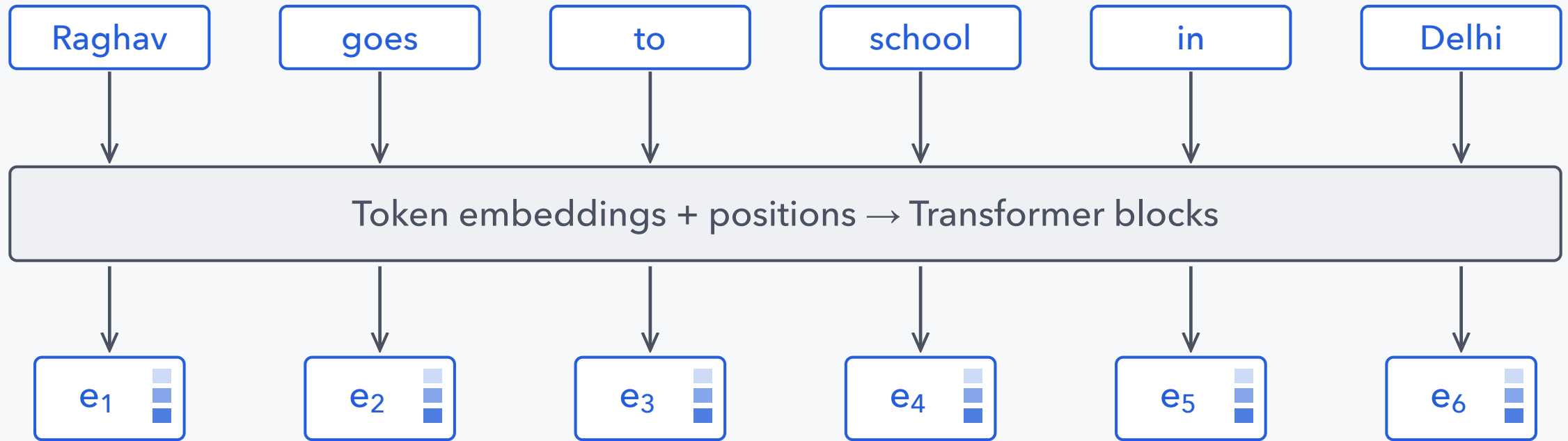
What is each token?

PERSON ... PLACE

Say it in Hindi

राघव दिल्ली में स्कूल जाता है।

Review: contextual token embeddings



e_i = final contextualised embedding · d numbers

E = final source embeddings stacked as rows · $6 \times d$

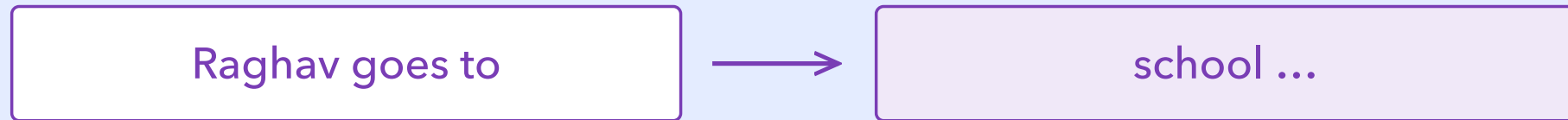
| The subscript tells us which token; the diagram shows its embedding after the blocks.

Attention, output representations and training targets

- 01 WHICH POSITIONS CAN ATTEND TO EACH OTHER?
- 02 WHICH EMBEDDINGS DOES THE TASK USE?
- 03 WHAT IS THE TRAINING TARGET?

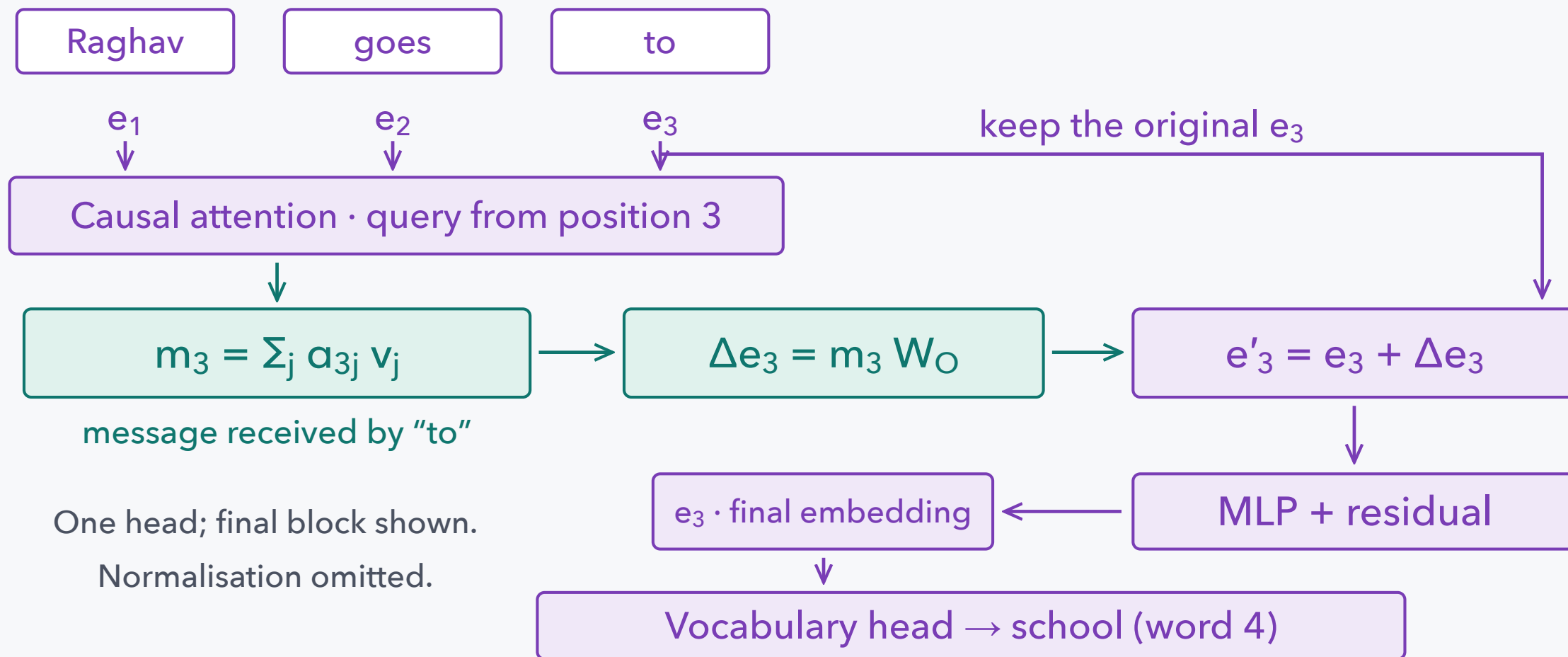
Part I · Generate the next token

Review of the causal Transformer.



Predict the next token from the available prefix.

Review: predicting the next token



m_3 updates token 3 ("to"); its final embedding predicts token 4 ("school").

Causal attention: current and earlier positions

Raghav goes to school in Delhi.

Raghav can read...

Raghav

× goes

× to

× school

× in

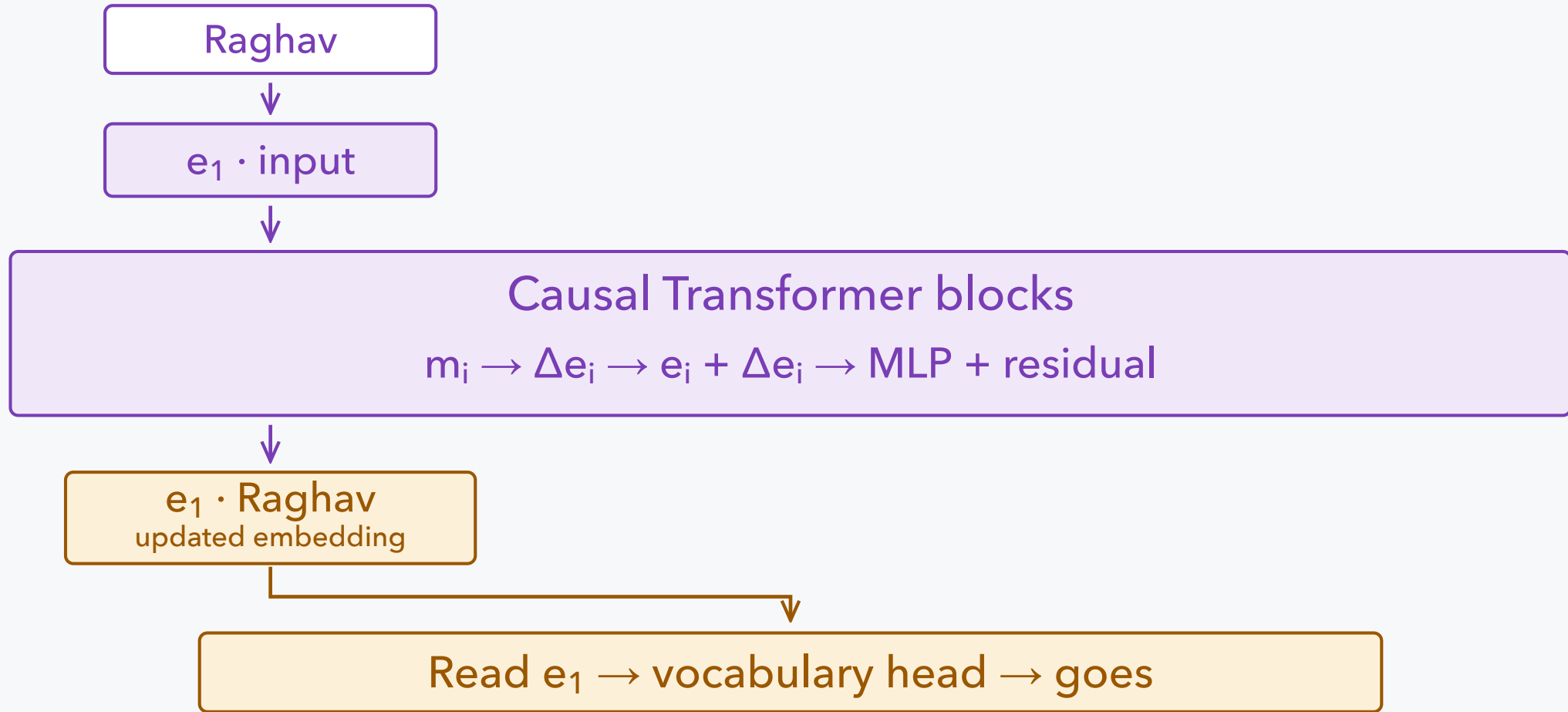
× Delhi

	key columns					
	R	g	t	s	i	D
R						
g						
t						
s						
i						
D						

6 × 6 · allowed connections

Filled cell = connection allowed, not an attention weight.

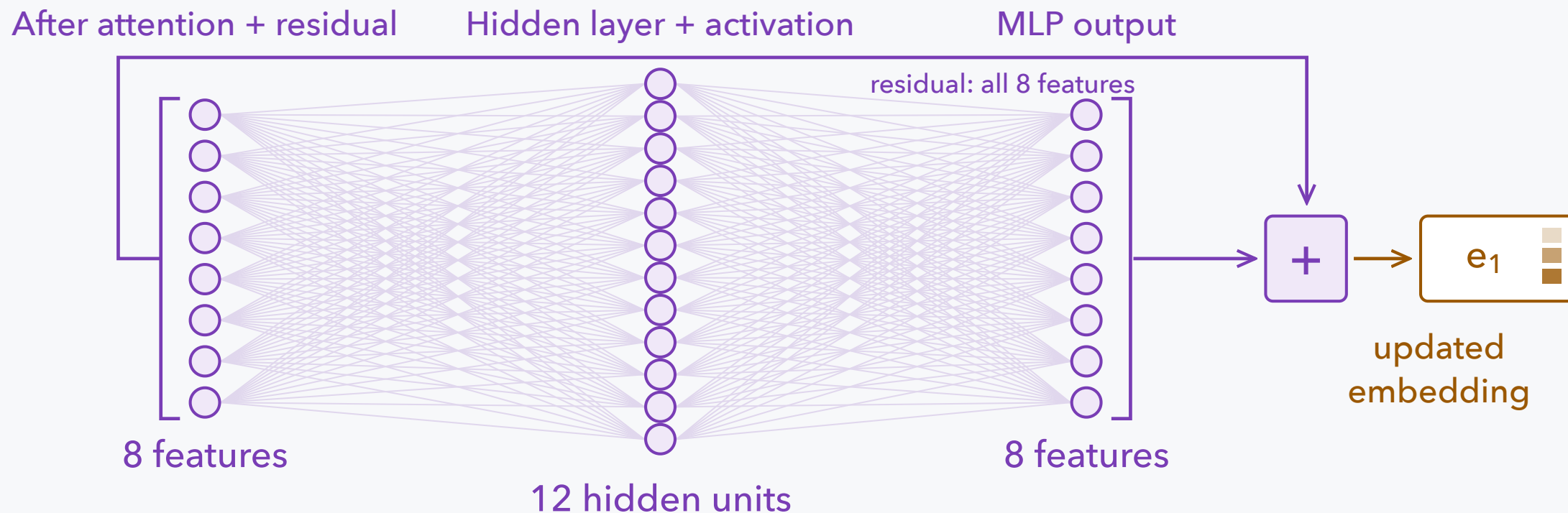
Generating a sequence one token at a time



Each word has an updated embedding. Read the last one to predict the next word.

The hidden layer inside a Transformer block

Zoom into the MLP at Raghav's position in the last block

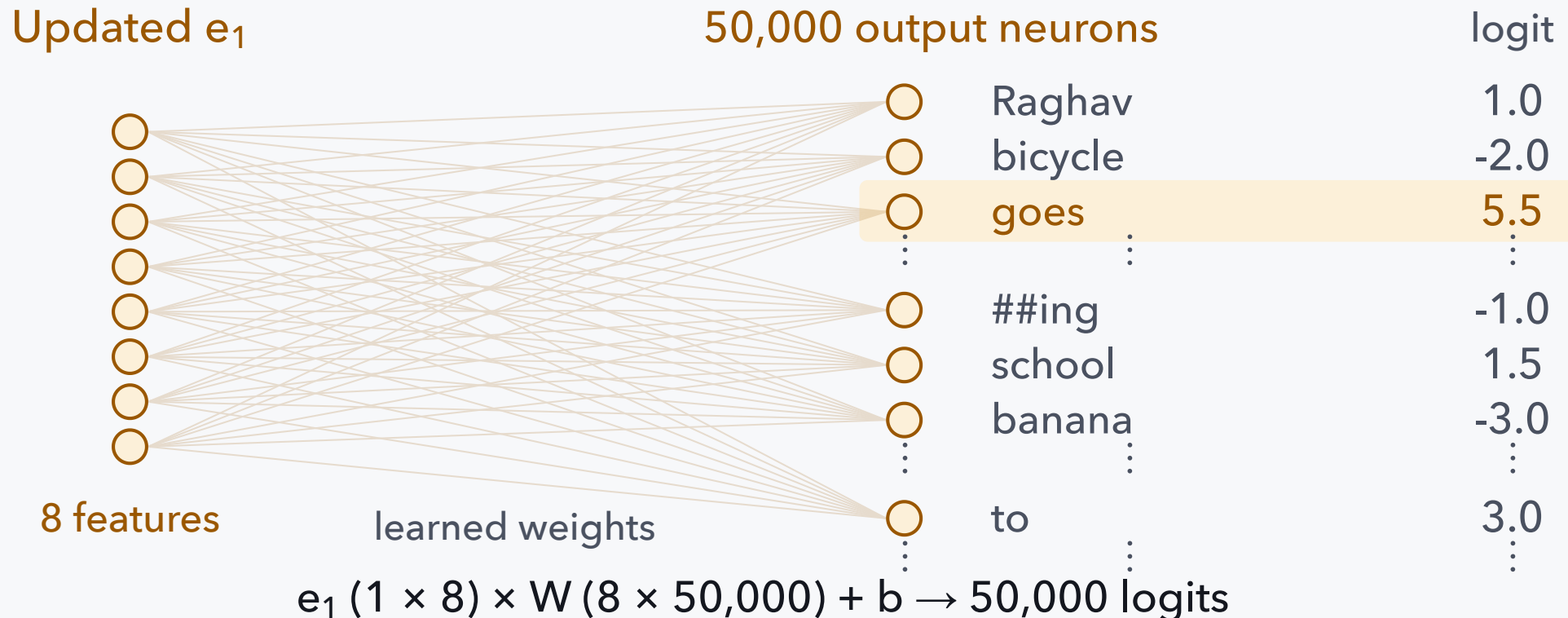


Example widths: $8 \rightarrow 12 \rightarrow 8$. The stack repeats attention and MLP updates.

| The MLP updates the embedding; the vocabulary layer reads it afterward.

One output neuron for every vocabulary token

Example: $d = 8 \cdot \text{vocabulary size } |V| = 50,000$



Selected entries shown; the dots stand for thousands of other tokens.

Every vocabulary token gets a score, including tokens unrelated to the input.

From vocabulary scores to the next token

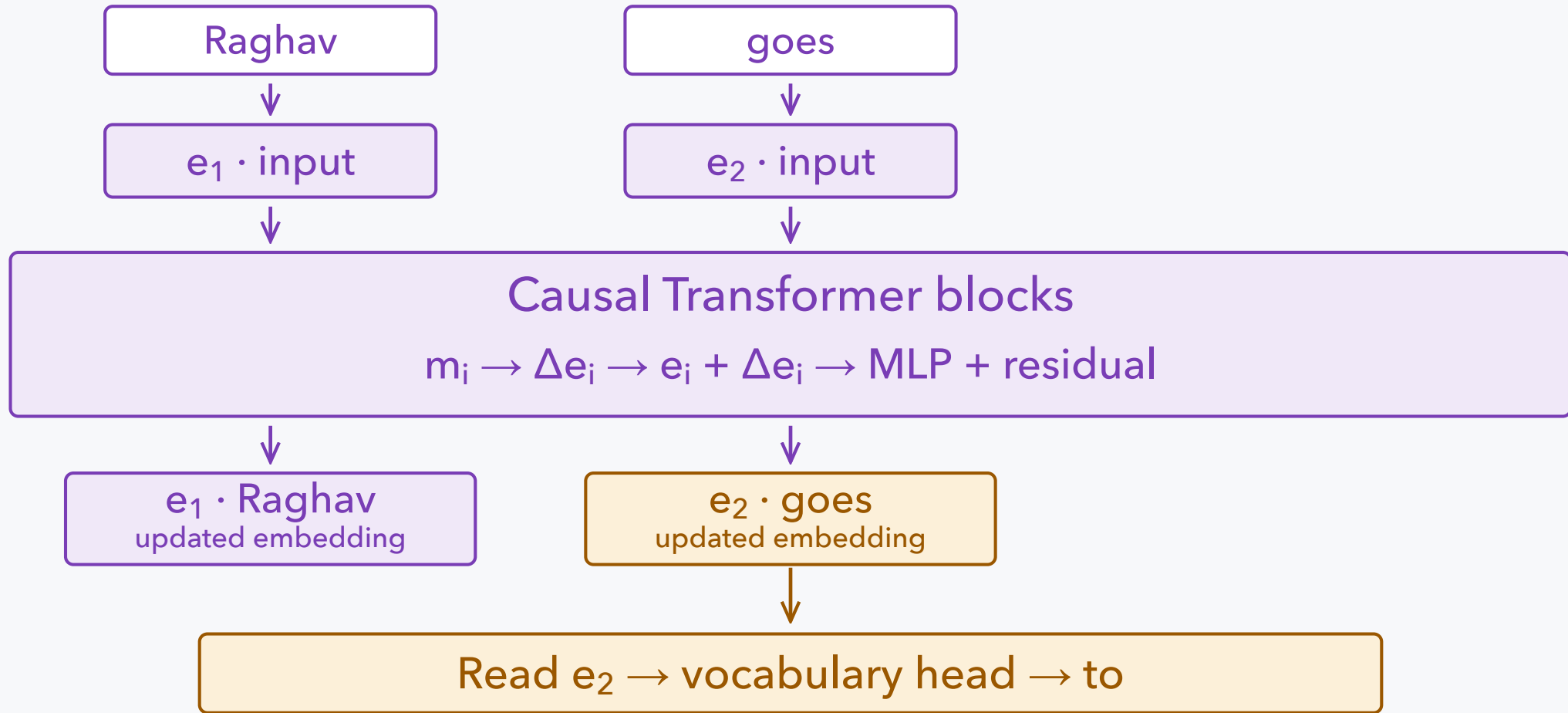
Softmax over all 50,000 vocabulary scores

token	logit	probability	
Raghav	1.0	0.94%	
bicycle	-2.0	0.05%	
goes	5.5	84.58%	→ Highest: goes
⋮	⋮	⋮	↓
##ing	-1.0	0.13%	
school	1.5	1.55%	
banana	-3.0	0.02%	
⋮	⋮	⋮	
to	3.0	6.94%	
Other 49,993 tokens combined:		5.80%	

$$p_j = \exp(\text{score}_j) / \sum \exp(\text{scores}) \cdot \text{sum over all 50,000 tokens}$$

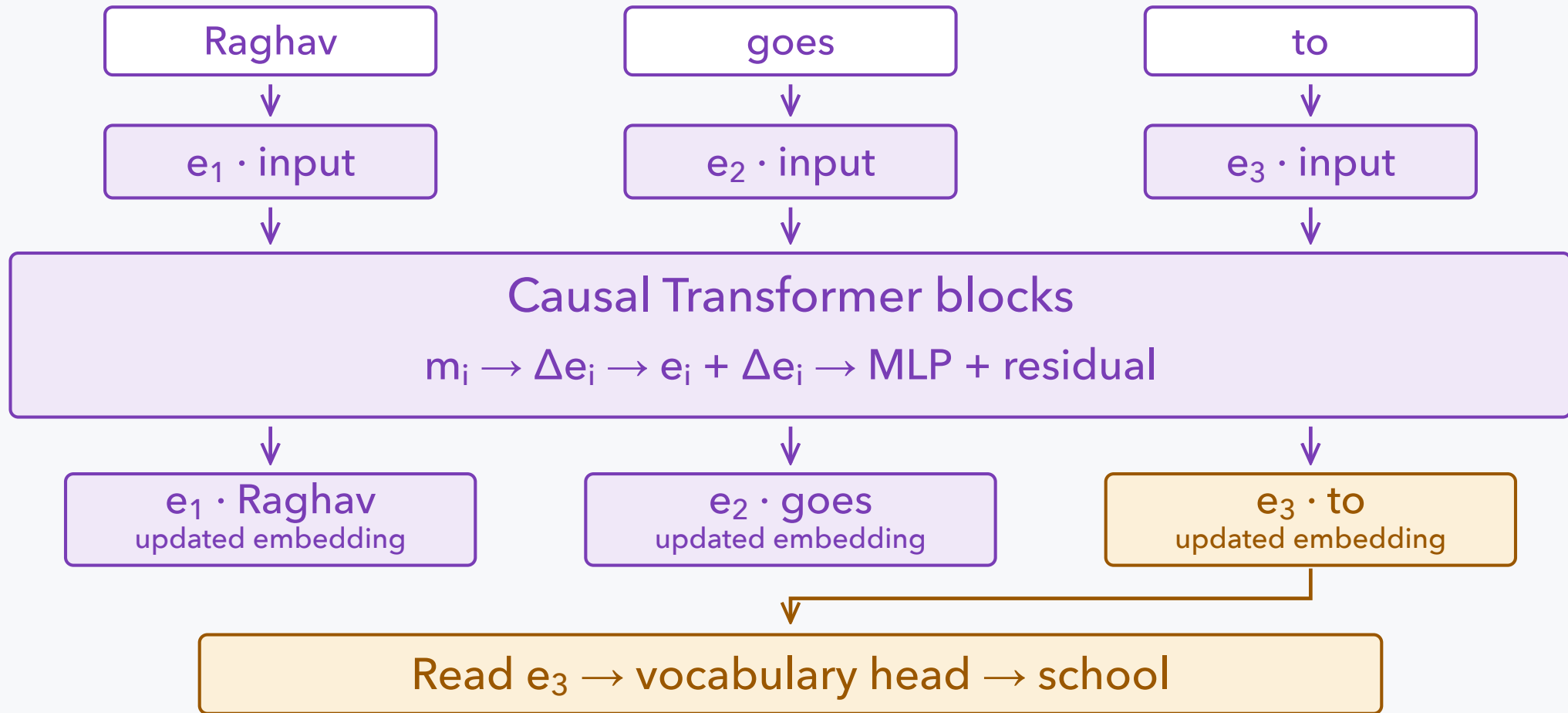
Greedy decoding selects the highest probability across the full vocabulary.

Generating a sequence one token at a time



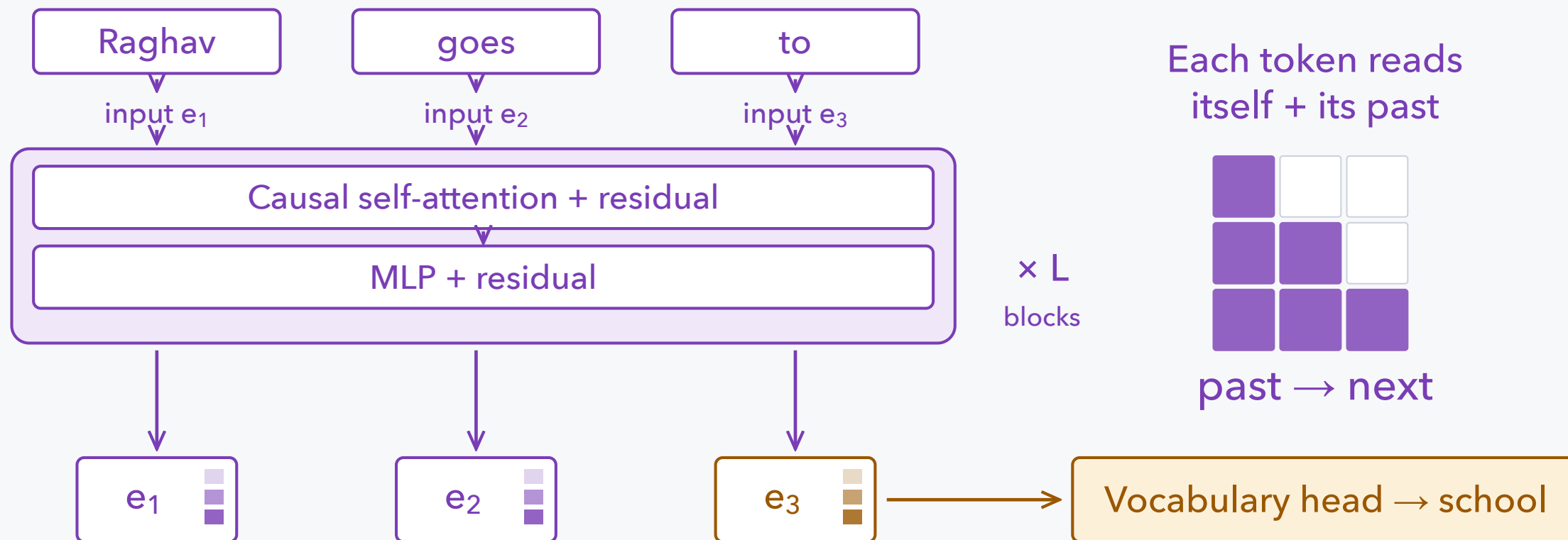
Each word has an updated embedding. Read the last one to predict the next word.

Generating a sequence one token at a time



Each word has an updated embedding. Read the last one to predict the next word.

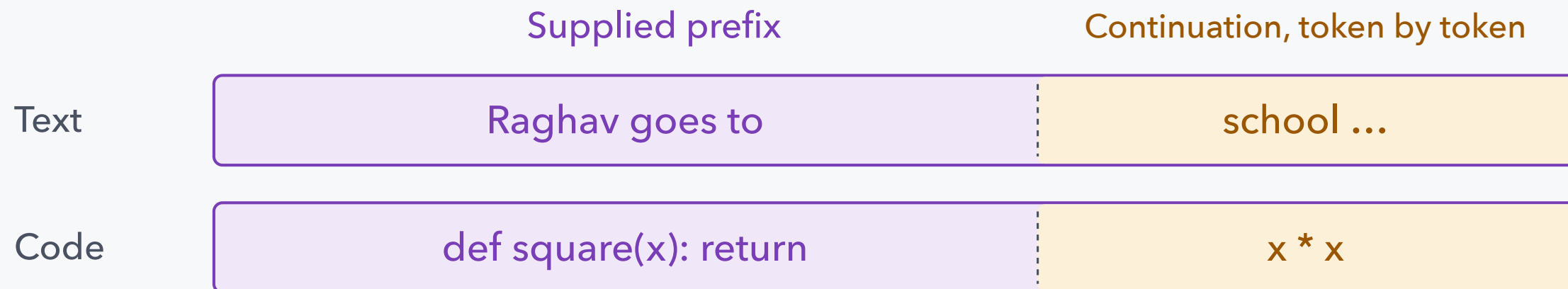
Decoder-only models: next-token prediction



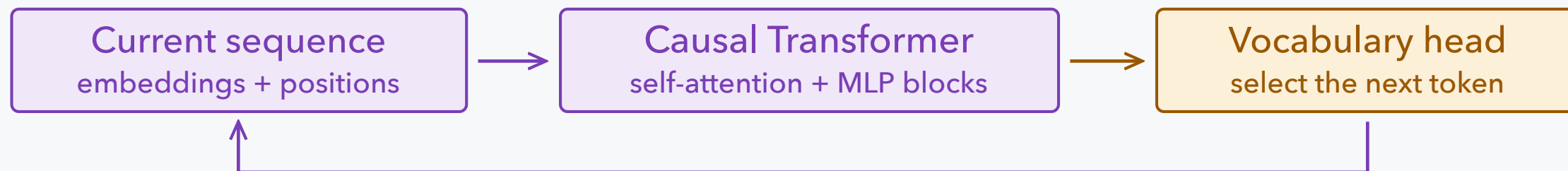
Read the last embedding. Append school. Repeat.

At each step, read the last updated embedding to predict the next token.

Decoder-only examples: text and code completion



At each step: the prefix and tokens generated so far form one sequence.



Append the selected token to the same sequence.

Q, K and V come from this sequence; there is no separate source encoder.

| One sequence, one causal Transformer, one next-token prediction at each step.

Part II · The whole input is available

Predict labels for a complete input sentence.



Use contextual embeddings to classify the input.

01 · Generation

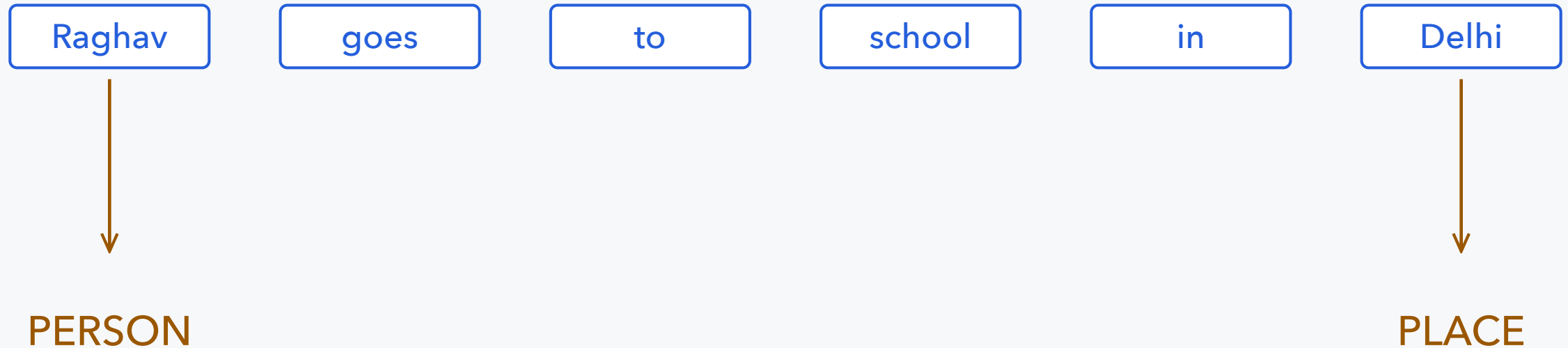
02 · Input labels

03 · Translation

04 · Cross-attention

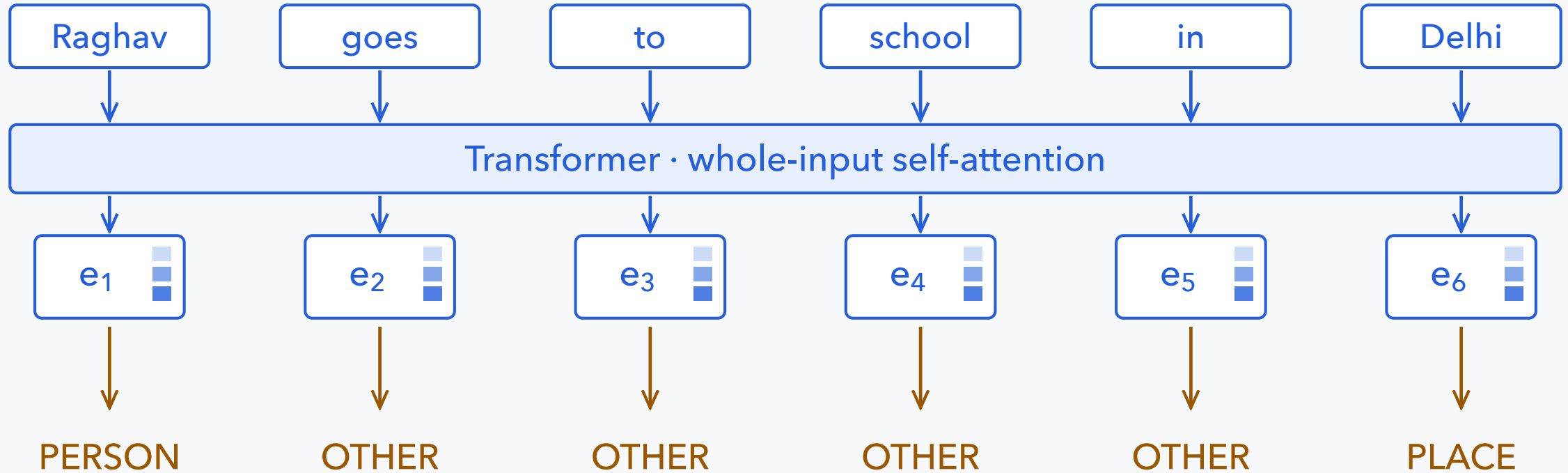
05 · Model families

Which words refer to people or places?



| Assign entity labels to words in the complete input.

Predicting a label at each input position



Do we need to generate a new sentence?

Six input positions → six label decisions.

Self-attention over the complete input

Raghav goes to school in Delhi.

Raghav can read...

Raghav goes
to school
in Delhi

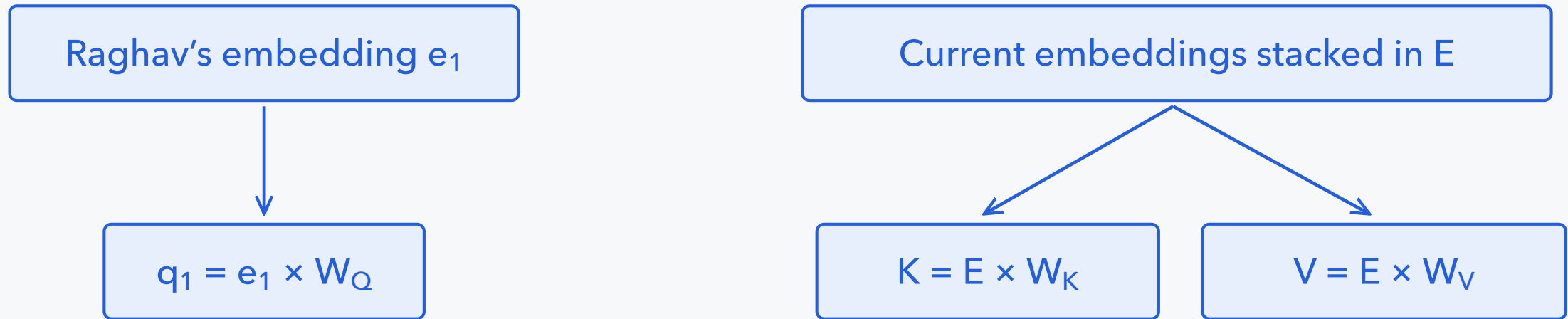
key columns

	R	g	t	s	i	D
R						
g						
t						
s						
i						
D						

6×6 · allowed connections

| Filled cell = connection allowed, not an attention weight.

Queries, keys and values in an encoder

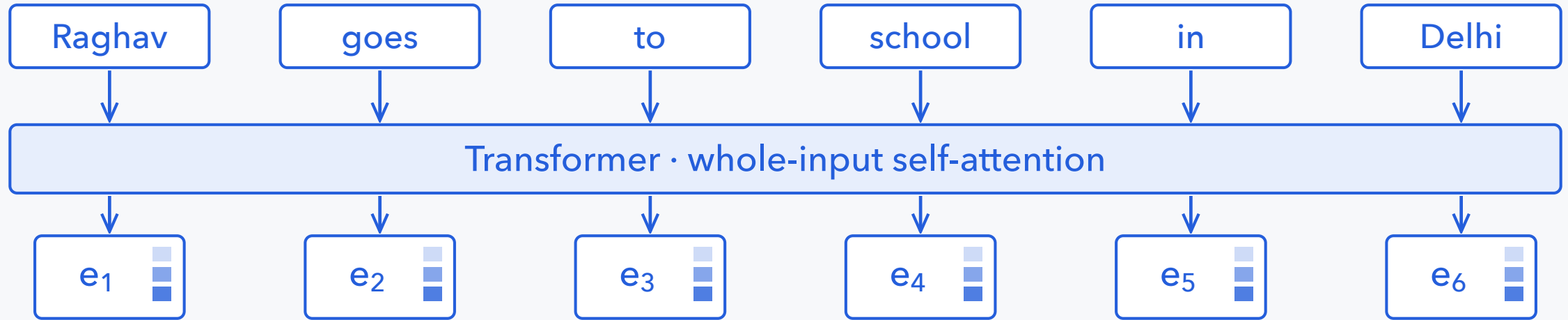


$$m_1 = \text{Attention}(q_1, K, V) = \text{softmax}(q_1 K^T / \sqrt{d_k}) V$$

The projections are shared across positions within a head.

| The attention calculation is unchanged; each position can now use the complete input.

One contextual vector per token



Raghav in context

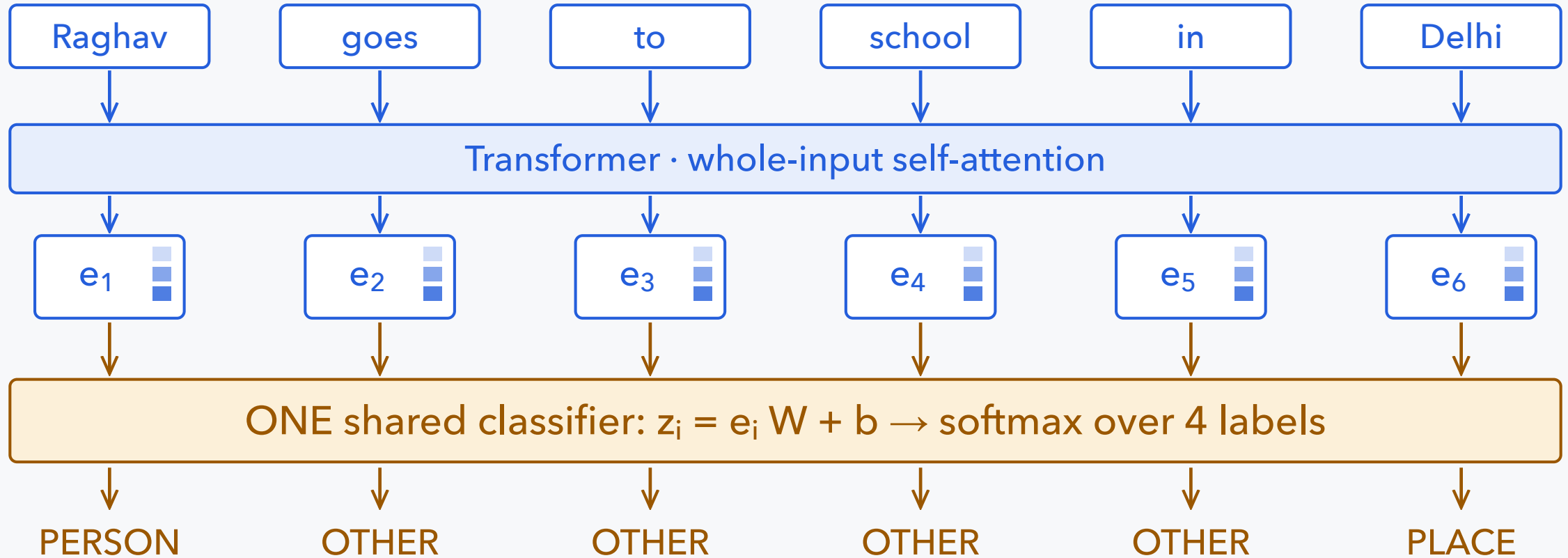
Delhi in context

$$\mathbf{E} \in \mathbb{R}^{6 \times d}$$

6 token positions $\times$ d features

| $\mathbf{E}$ has shape $n \times d$.

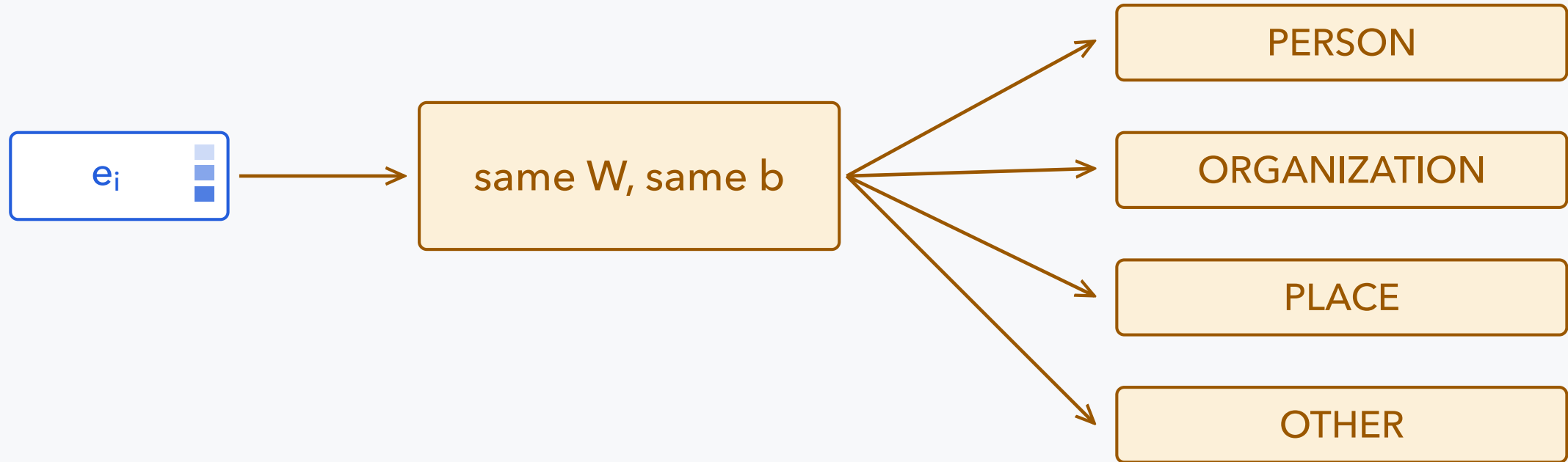
A shared classifier for all token positions



Entity labels at every position: PERSON · ORGANIZATION · PLACE · OTHER

| One W and b , reused at all six positions.

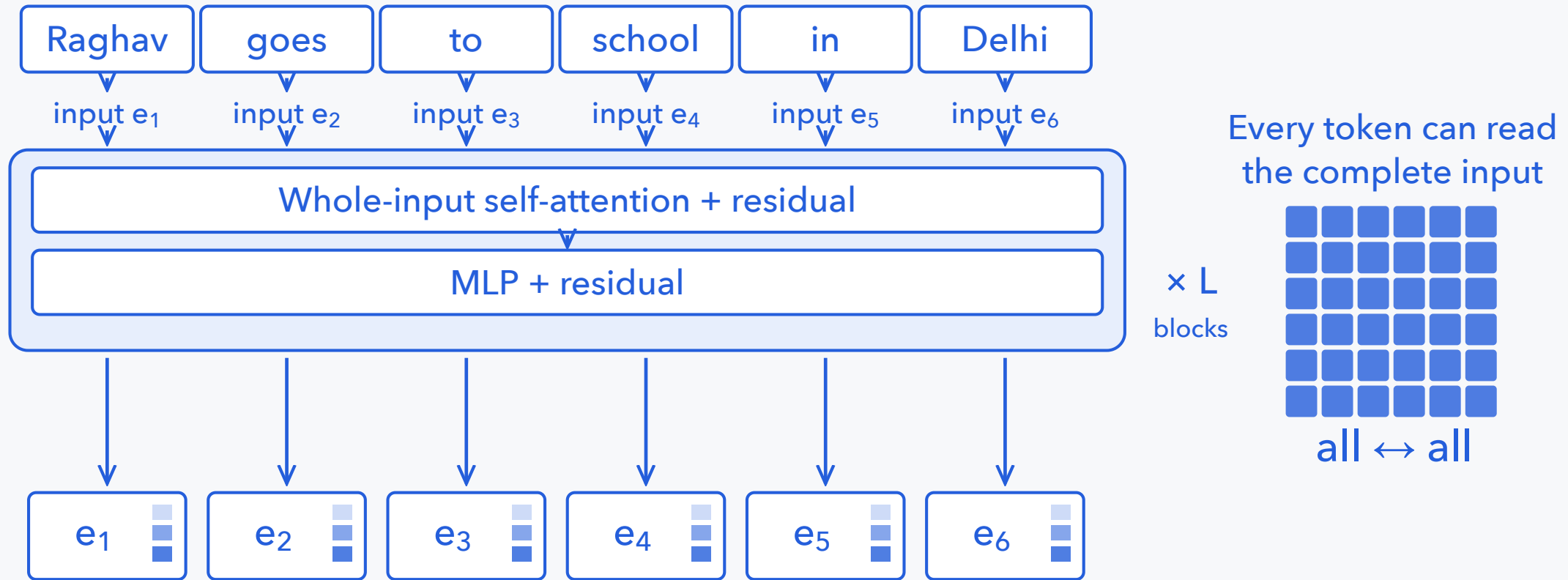
Four entity labels for each token



d features $\rightarrow$ 4 scores $\rightarrow$ 4 probabilities

| Training target: one entity label per position.

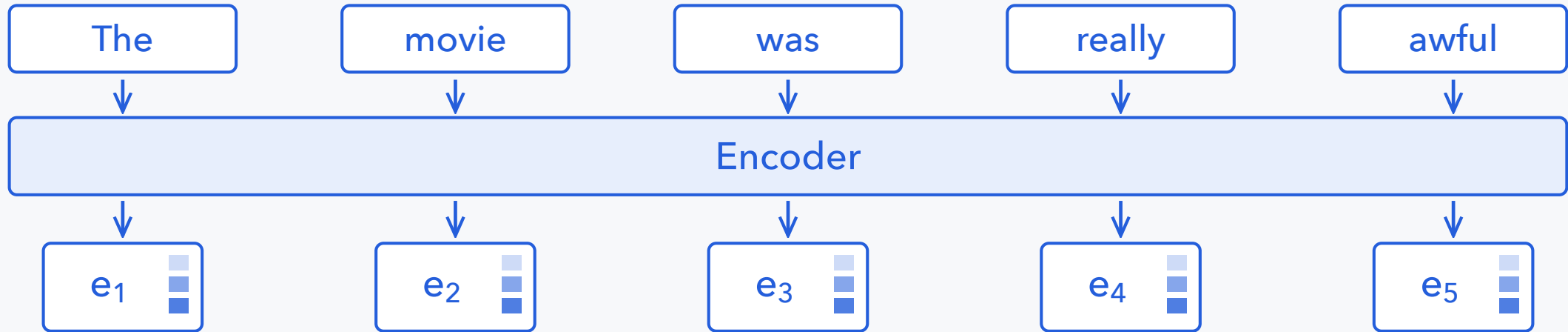
Encoder: a contextual representation for each token



Six input tokens → six contextual embeddings.

Use the embeddings for token labels, sentence readouts or similarity.

Predicting one label for a sentence

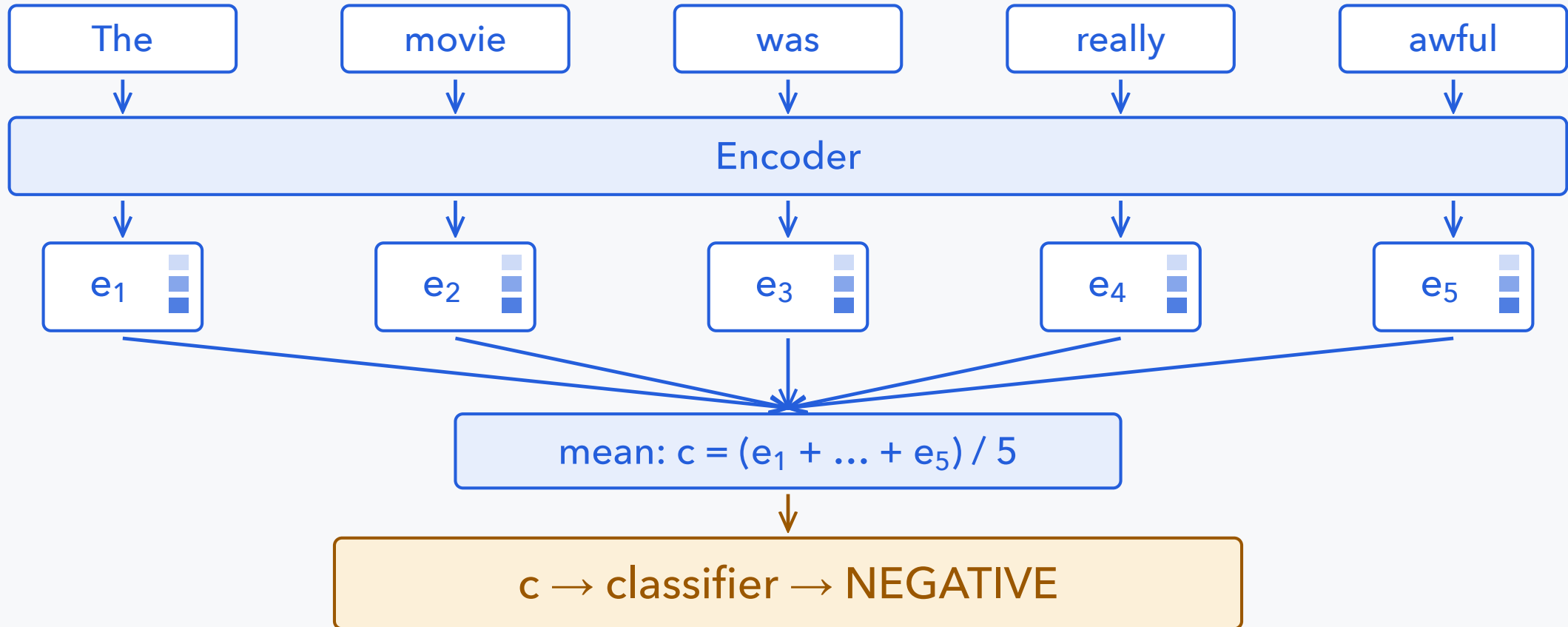


Which state should we read?

One review → one sentiment label

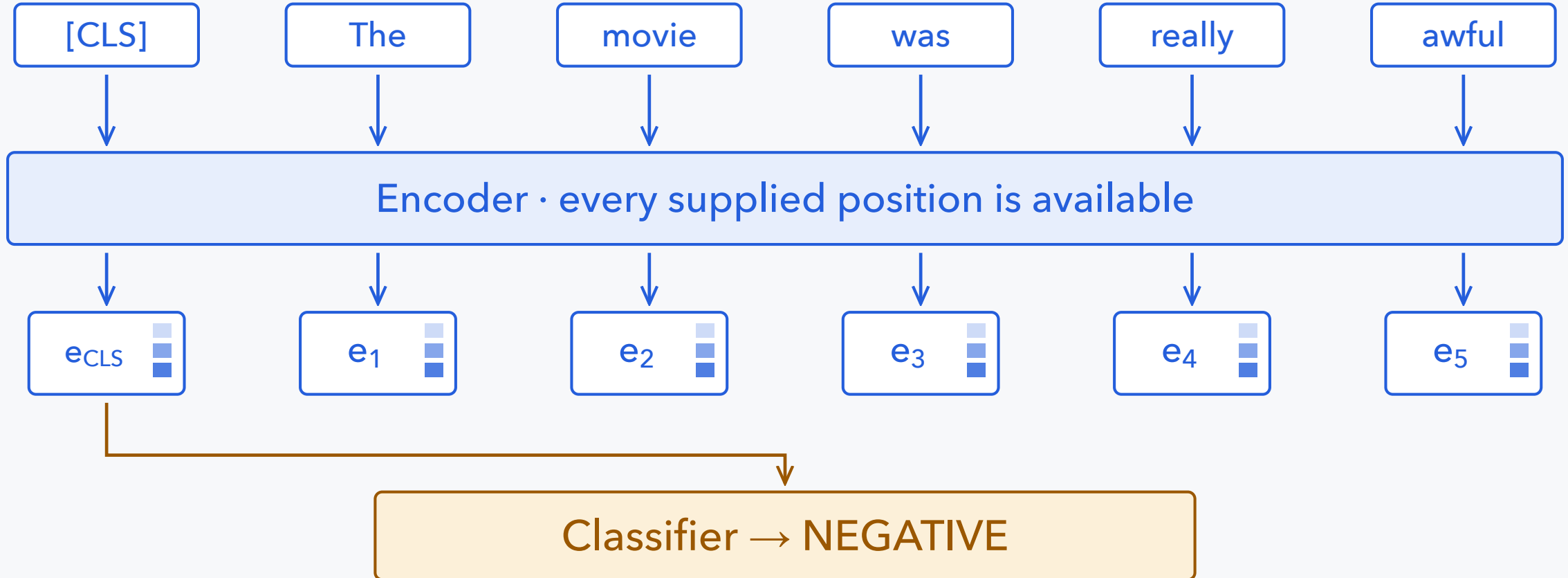
The movie was really awful. → NEGATIVE

Sentence classification with mean pooling



| The pooled vector is the input to the sentence classifier.

Sentence classification with a CLS token



[CLS] participates in ordinary whole-input attention.

Where does the CLS embedding come from?

Embedding table · trainable

token ID	d numbers
[CLS]	
Raghav	
goes	

[CLS] has its own row, like a word.

How does that row start?

From scratch: small random values, once.

Pretrained: load the saved row.

Look up e_{CLS} , then add position

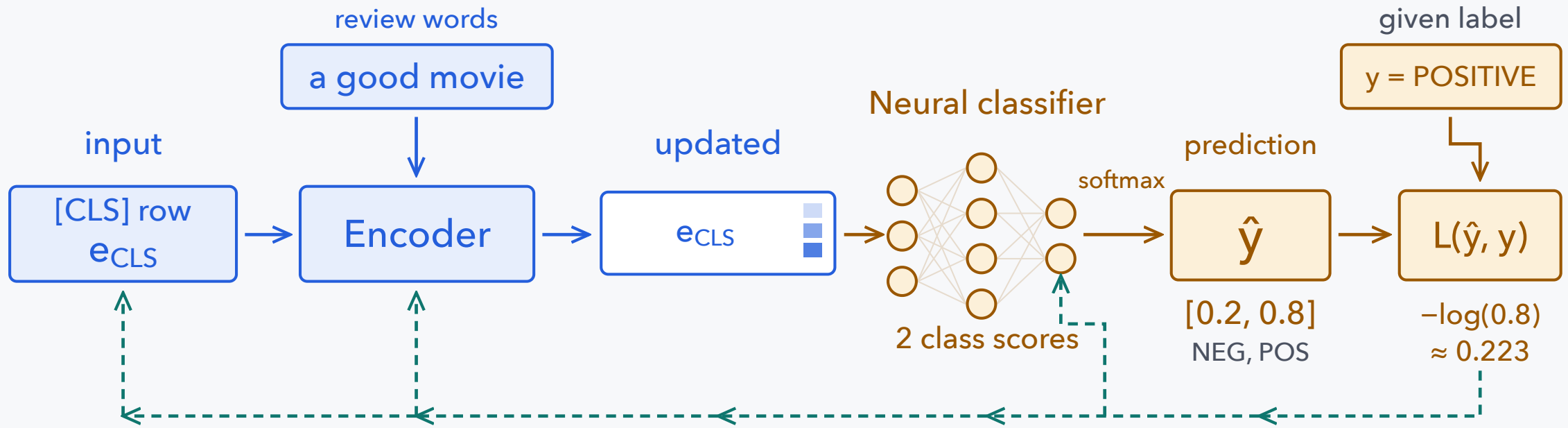
Encoder + sentence $\rightarrow$ updated e_{CLS}

Same table row for every sentence; the encoder output depends on the sentence.

[CLS] has a token ID and a trainable row in the embedding table.

CLS learns through the prediction loss

FORWARD PASS → compute the prediction and loss



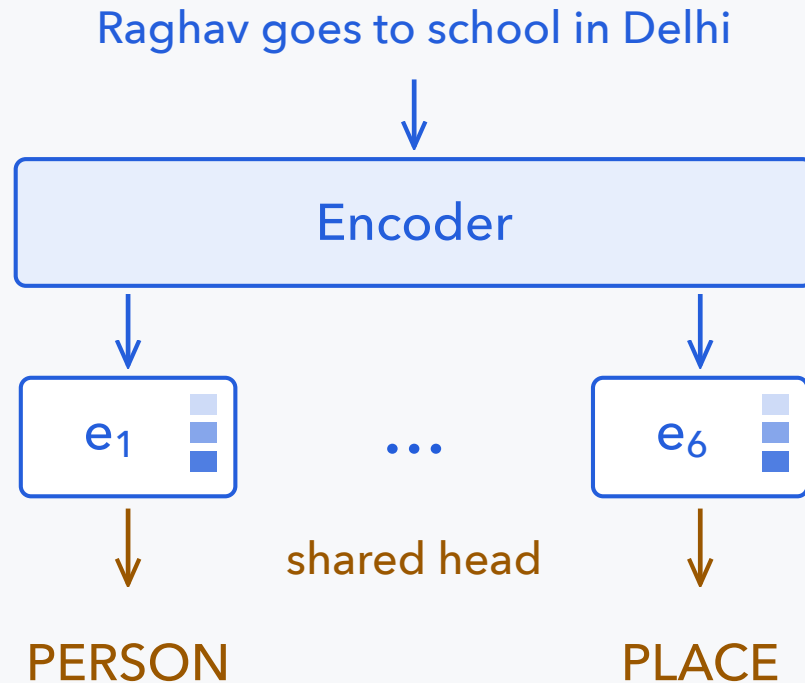
← BACKWARD PASS: loss gradients flow through the same operations

Optimizer updates: CLS table row + encoder + classifier parameters.

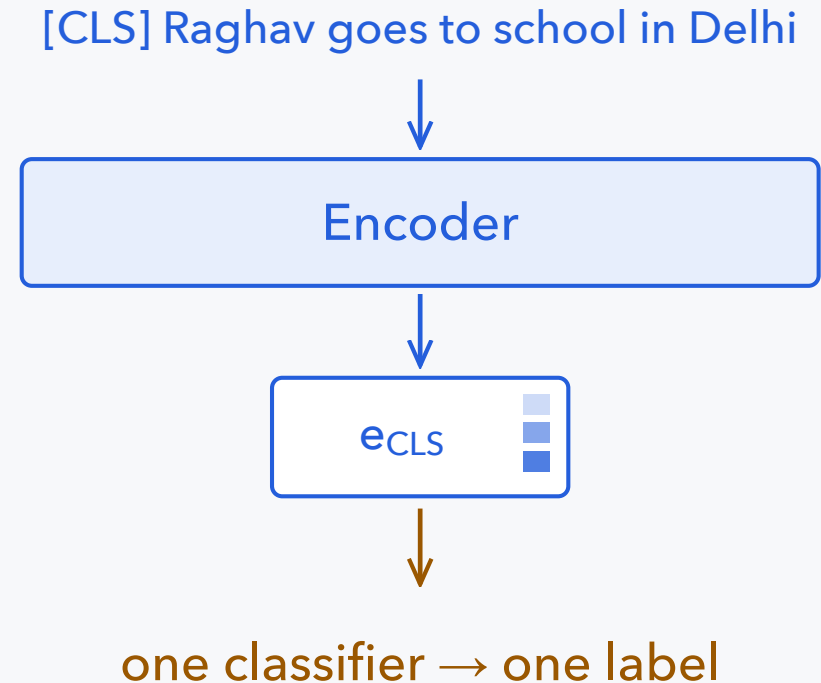
| Compare prediction $\hat{y}$ with label y ; use the loss to train the CLS representation.

Token classification and sentence classification

A LABEL PER TOKEN



ONE SENTENCE LABEL



| Read every state for NER; read CLS for one sentence label.

Six words, eight features per word

Raghav goes to school in Delhi

Encoder · choose $d = 8$ for this example

	1	2	3	4	5	6	7	8
Raghav								
goes								
to								
school								
in								
Delhi								

6 rows = 6 token positions

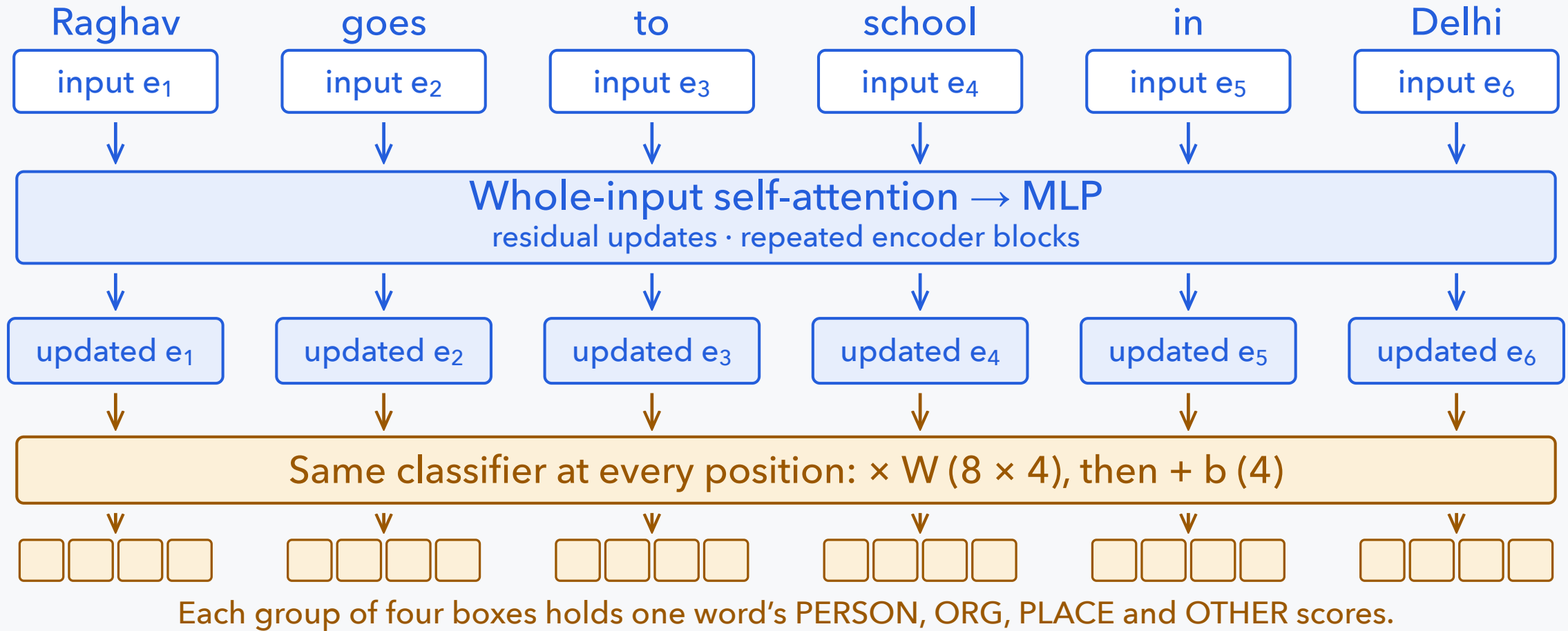
8 columns = 8 features

E has shape 6×8

Each row is one updated embedding. The cells stand for numbers.

| 6 counts token positions; 8 counts features inside each embedding.

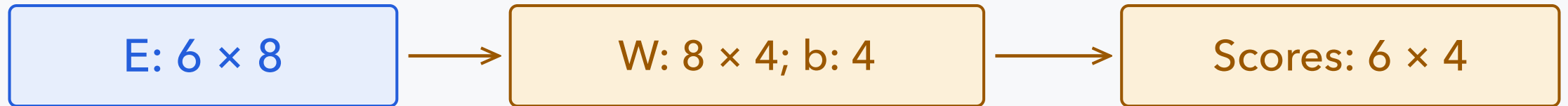
NER: follow each word from input to scores



6×8 input features $\rightarrow 6 \times 8$ updated features $\rightarrow 6 \times 4$ class scores.

NER: turn each set of scores into a label

One shared classifier reads each of the six embeddings



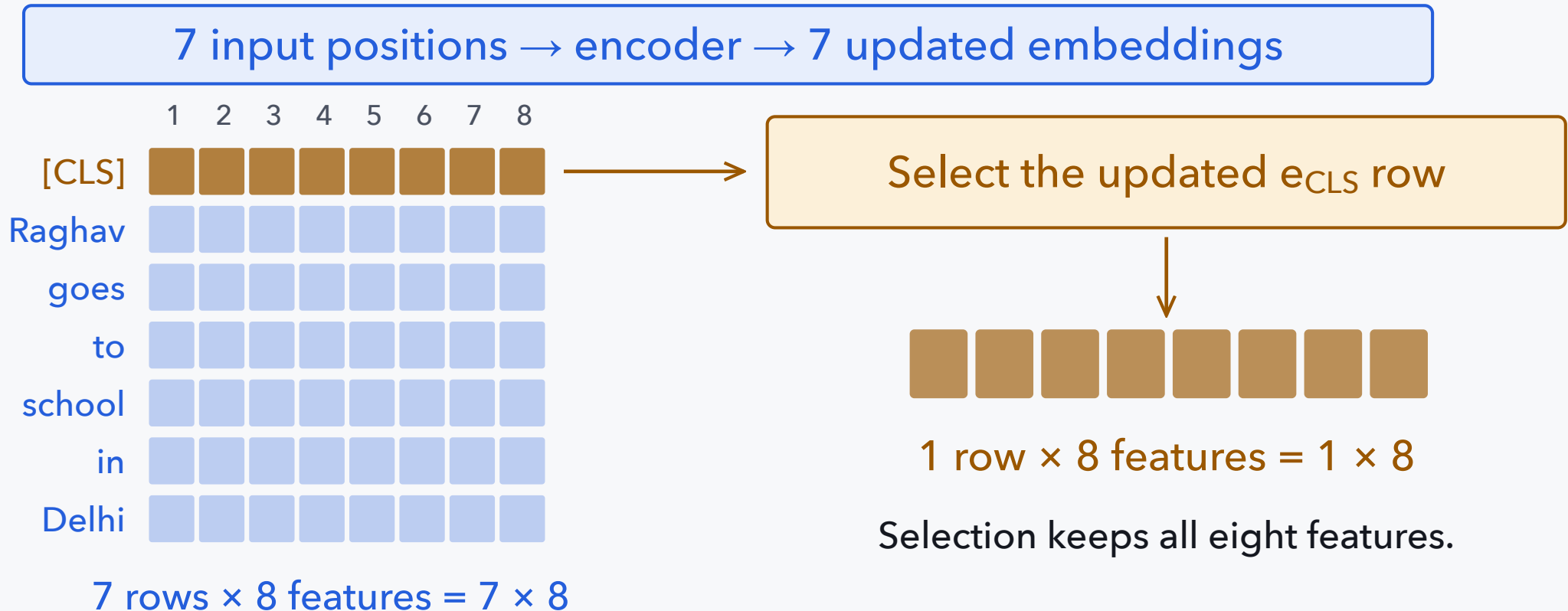
	PERSON	ORG	PLACE	OTHER	chosen label
Raghav					PERSON
goes					OTHER
to					OTHER
school					OTHER
in					OTHER
Delhi					PLACE

Softmax within each row → 4 probabilities → one label per word.

The classifier changes 8 features into 4 scores at every position.

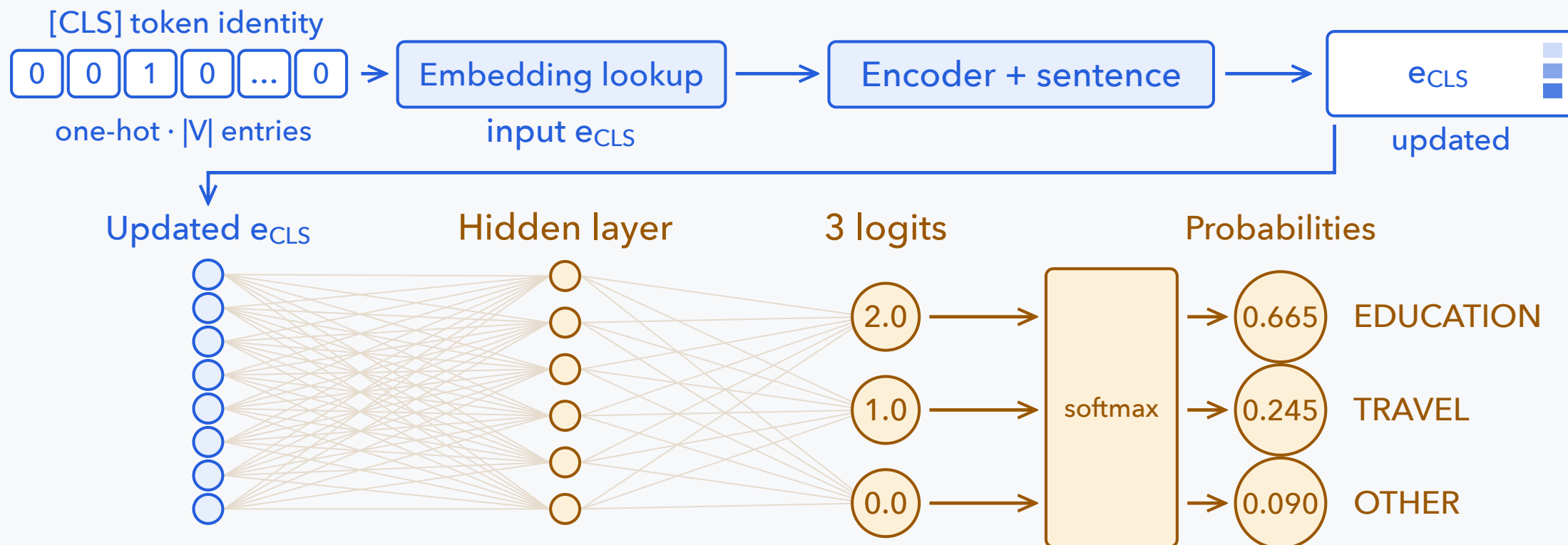
CLS: select one row from seven

Add [CLS] to the same six-word sentence



7 × 8 becomes 1 × 8 when we select the updated CLS embedding.

A neural classifier reads the updated CLS embedding

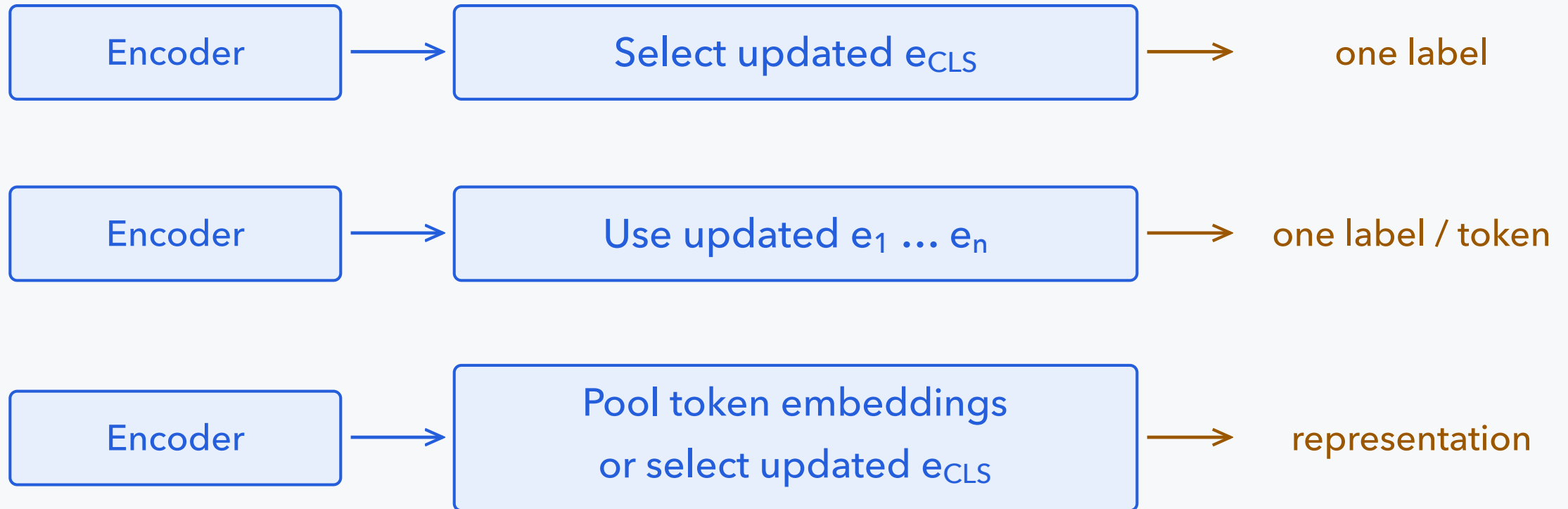


8 dense features → 6 hidden units → 3 logits → 3 probabilities

Example scores; the hidden layer has a nonlinear activation.

The one-hot vector identifies CLS; its updated embedding is used for classification.

One encoder, several ways to use its embeddings

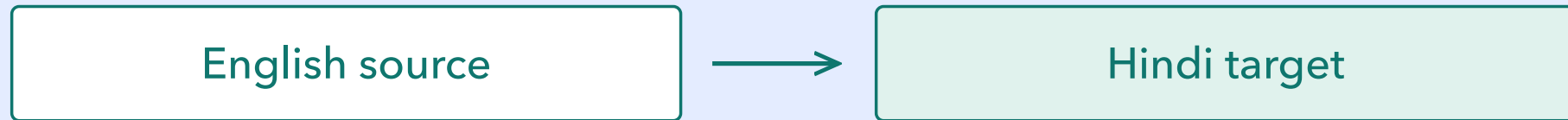


Readout = choosing or combining the encoder's updated embeddings.

| Selecting updated CLS and pooling token embeddings are two readout choices.

Part III · Sequence-to-sequence prediction

Encode the English input and generate a Hindi translation.



Represent the source and condition target generation.



Example: English-to-Hindi translation

Raghav goes to school in Delhi.



राघव दिल्ली में स्कूल जाता है।

Encode the source sentence and generate the target sequence.

| राघव दिल्ली में स्कूल जाता है।

How does Hindi generation begin?

[शुरू] means “begin generating” · a special token ID



Look up e_1 , add p_1 , then run the decoder with the English source.

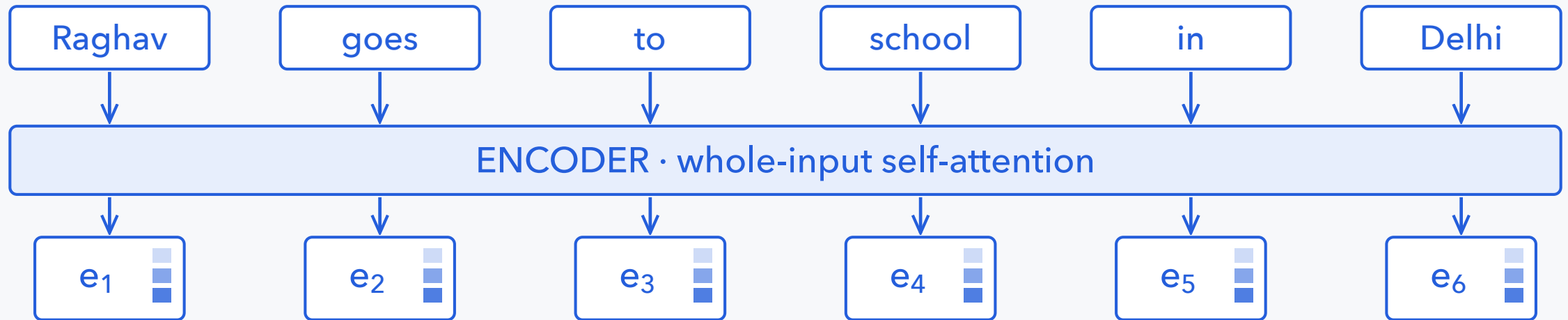
We supply [शुरू]. The first predicted Hindi word is राघव.

[शुरू] राघव दिल्ली में स्कूल जाता है [समाप्त]

The markers control generation; the translation contains only the six Hindi words.

[शुरू] starts the process; [समाप्त] tells us to stop.

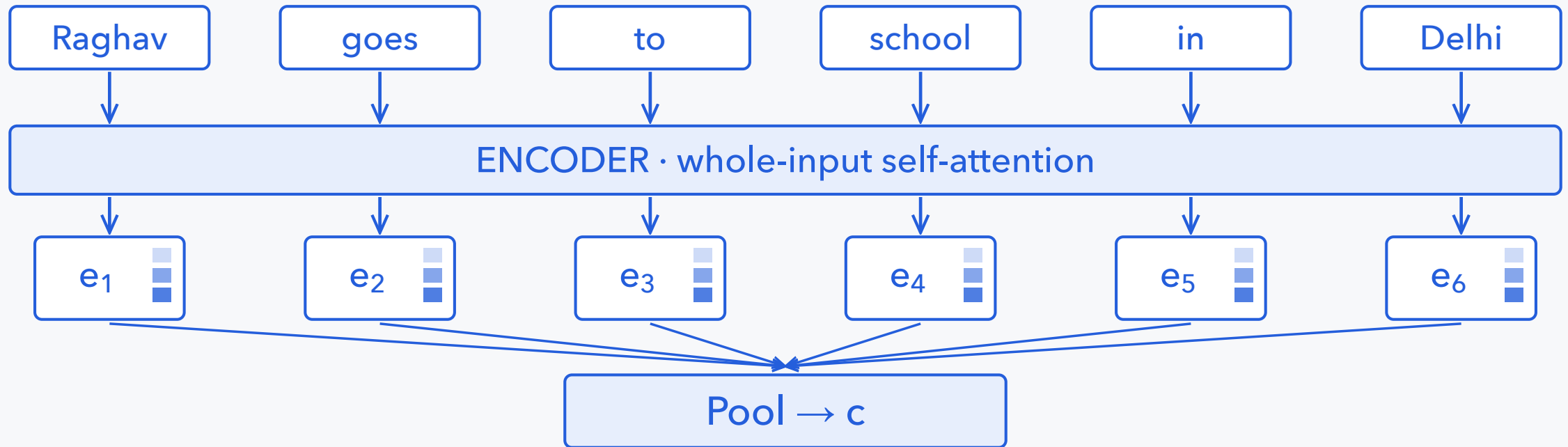
Step 1: encode the source sentence



Keep one contextual vector per source position.

| The same encoder states we used for NER.

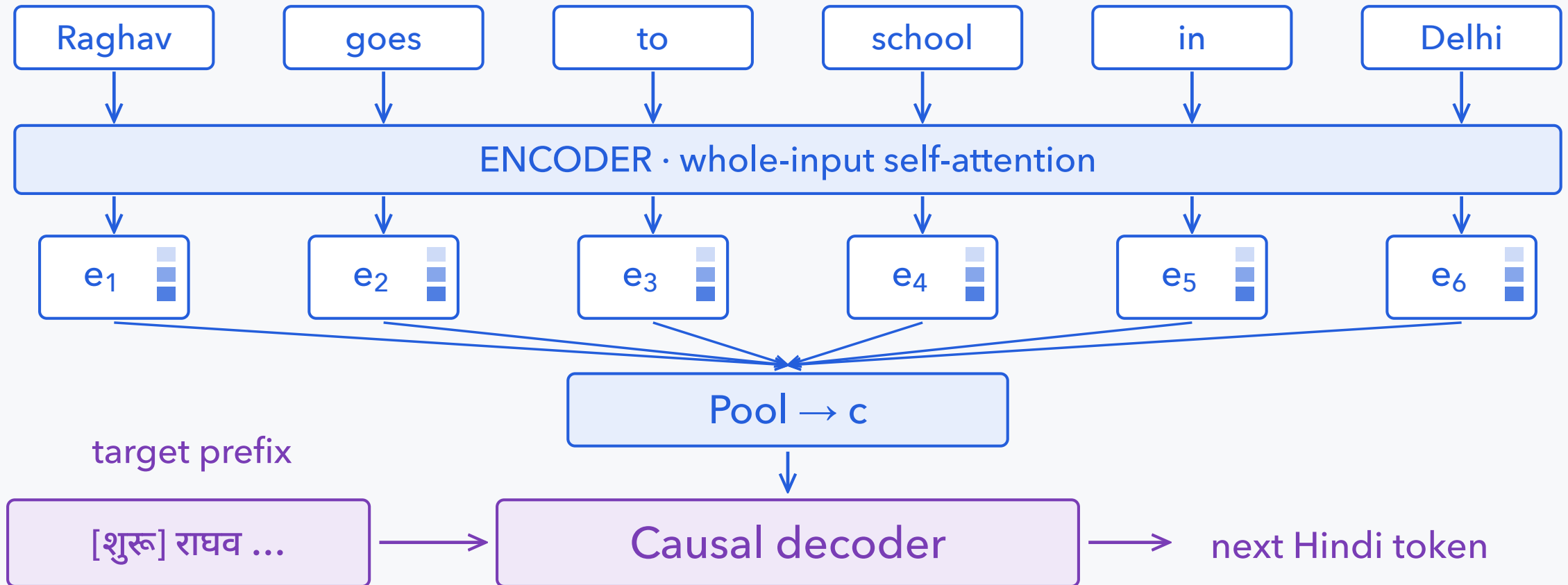
First attempt: compress the source into one vector



One fixed source vector, $c \in \mathbb{R}^d$

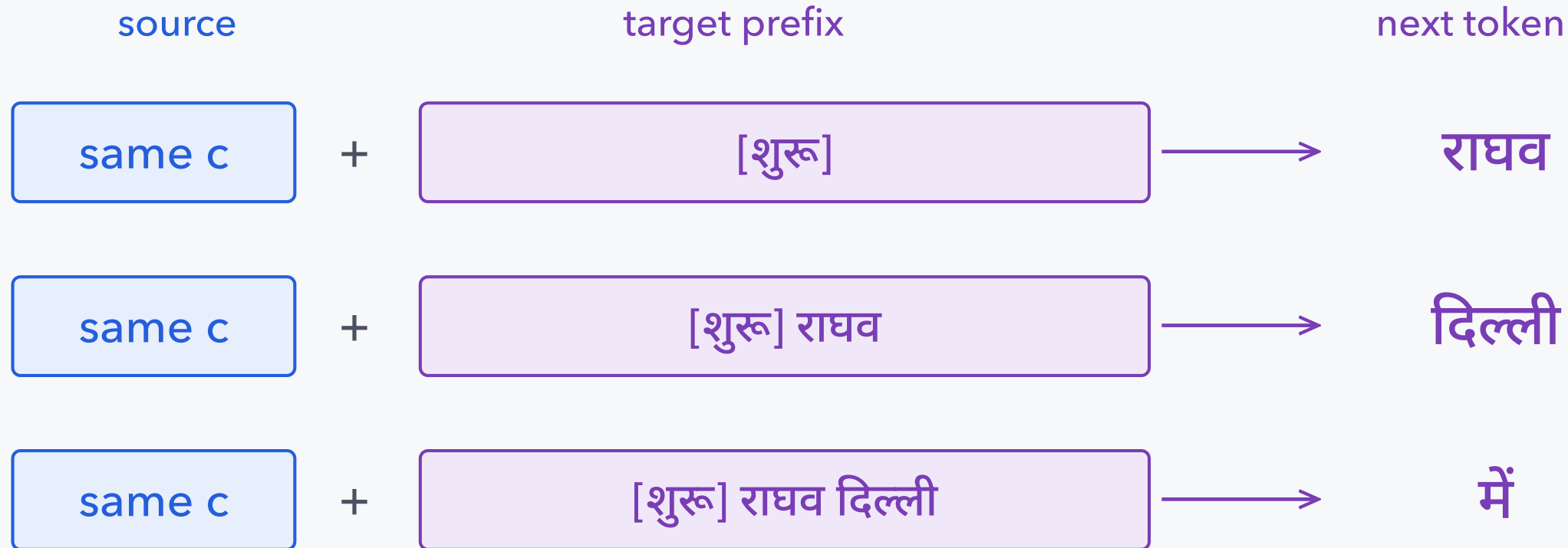
| $c = \text{Pool}(e_1, \dots, e_6)$

Conditioning the decoder on the source vector



| $p(y_t | y_{<t}, c)$

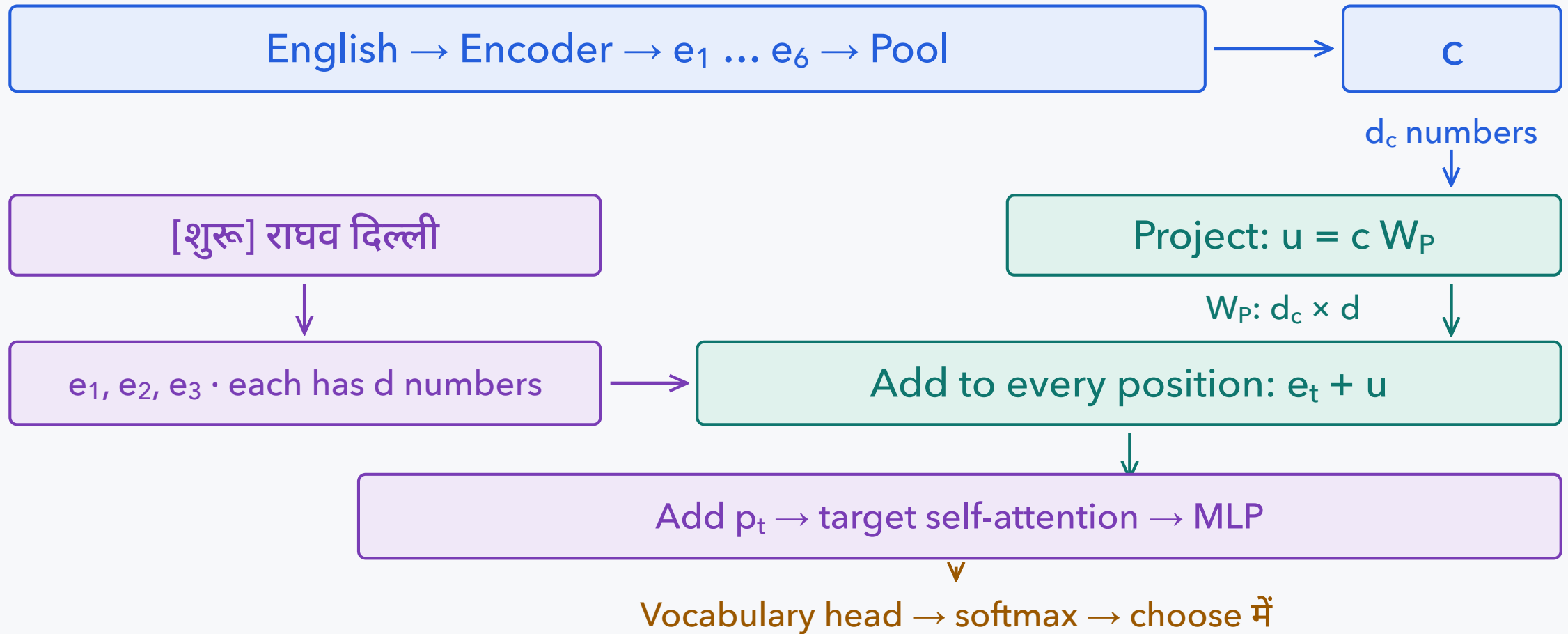
A fixed source vector at each generation step



$$p(y_t \mid y_{<t}, c)$$

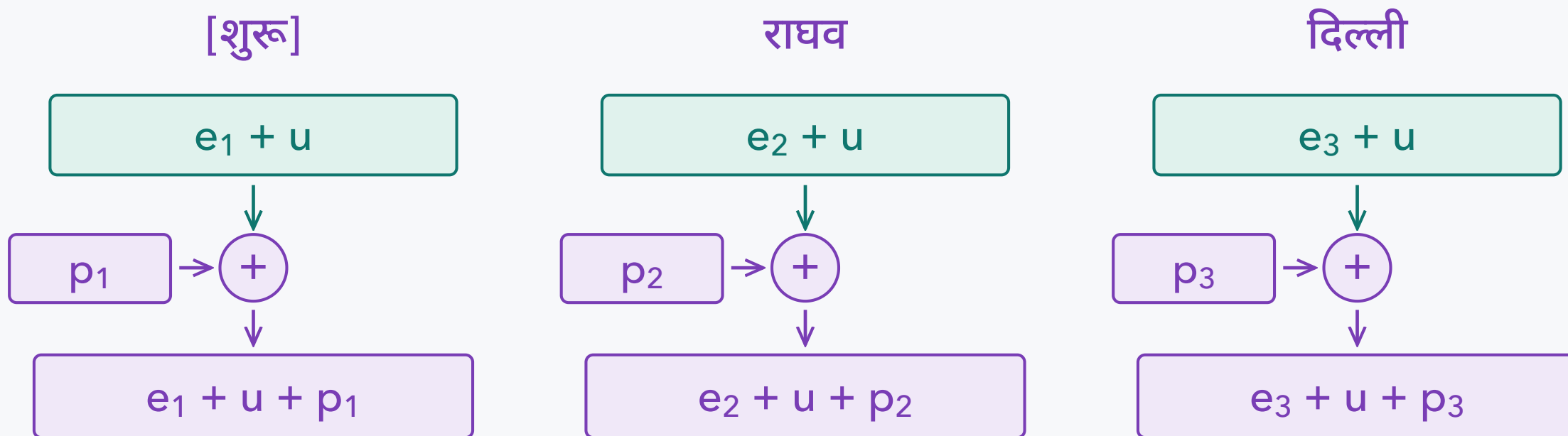
| The decoder has access to c at every step.

Option 1: add context to every target embedding



Project c to the decoder width, then broadcast the same vector across target positions.

Add a position vector at each target position



p_i is a position vector with d numbers, just like e_i and u .

Same source context u ; a different p_i for each position.

Add matching coordinates. The result still has d features.

e_i identifies the token; u carries source context; p_i marks the position.

Causal self-attention over the Hindi prefix

The same attention operation, now among Hindi target positions.

Keys / values: positions we can read

	[शुरू]	राघव	दिल्ली	में
[शुरू]	✓	×	×	×
राघव	✓	✓	×	×
दिल्ली	✓	✓	✓	×

Rows = queries · filled cells = allowed

Q, K, V all come from target e_i

[शुरू]

reads itself

राघव

reads शुरू and itself

दिल्ली

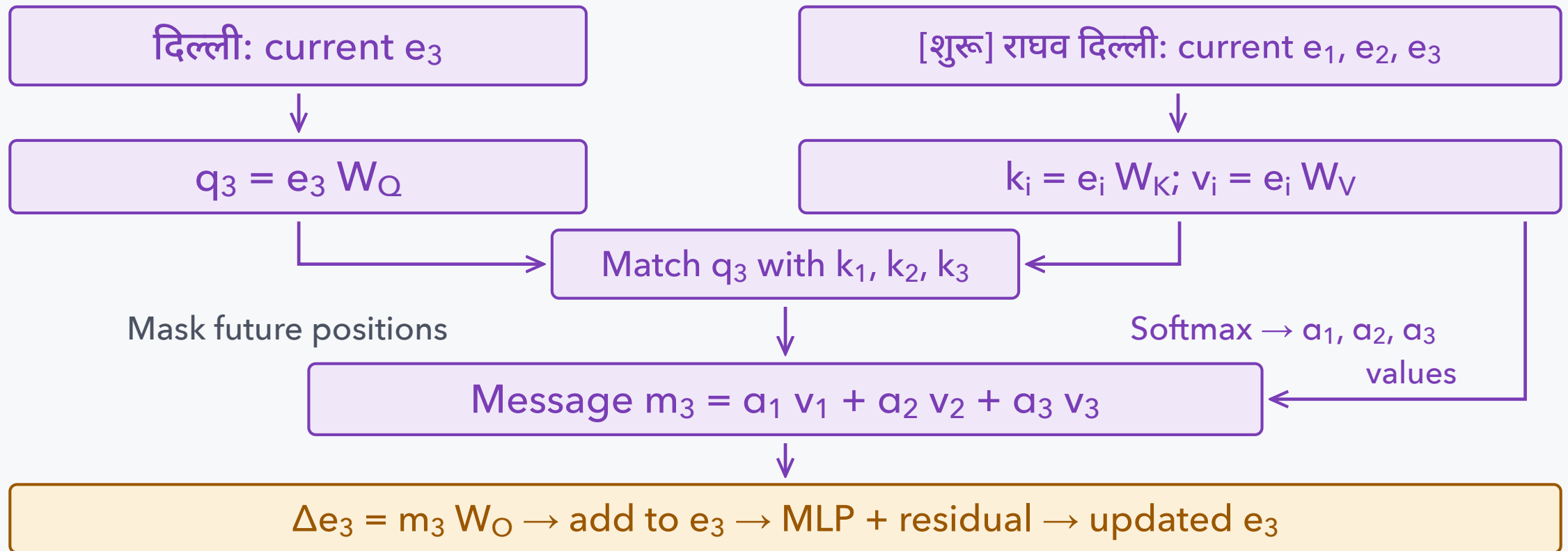
reads all three supplied positions

में is the next prediction. Its embedding is not an input yet.

Target queries read target keys and values. The causal mask keeps future tokens hidden.

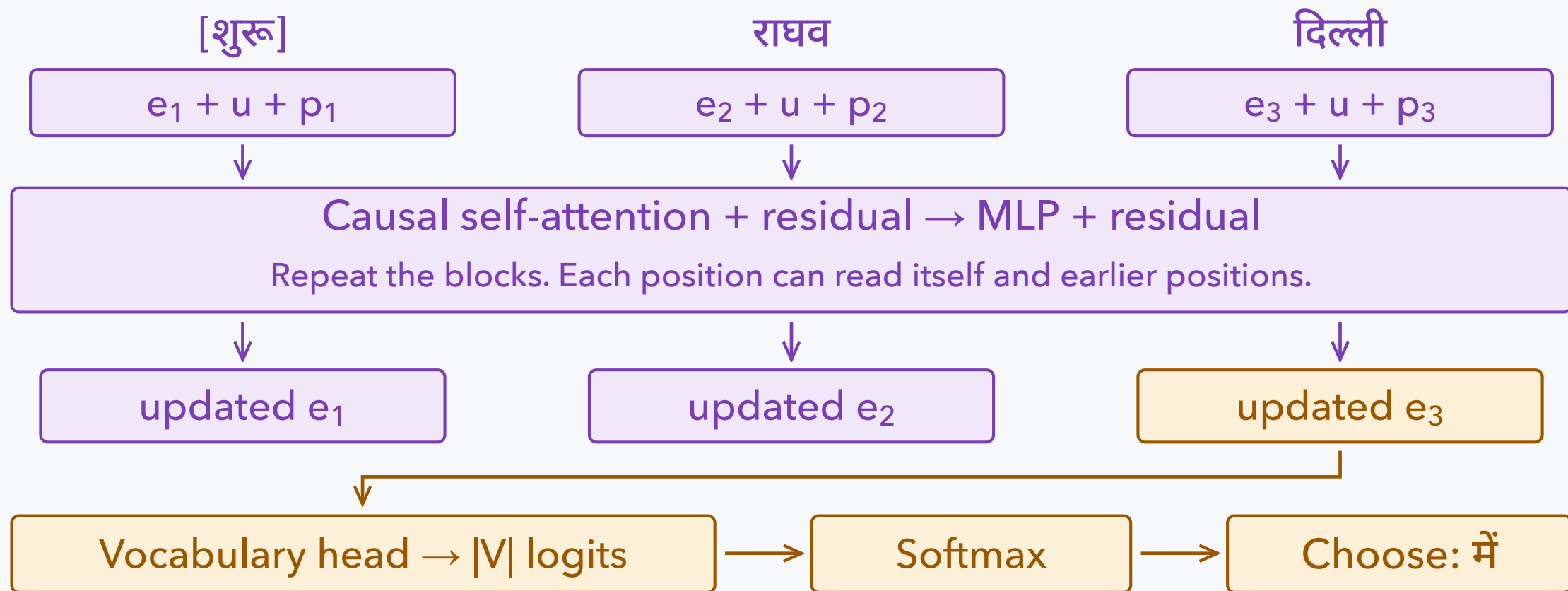
Updating the embedding at दिल्ली

Here e_i means the current target embedding, after adding u and p_i .



The message m_3 updates दिल्ली; its final embedding will predict में.

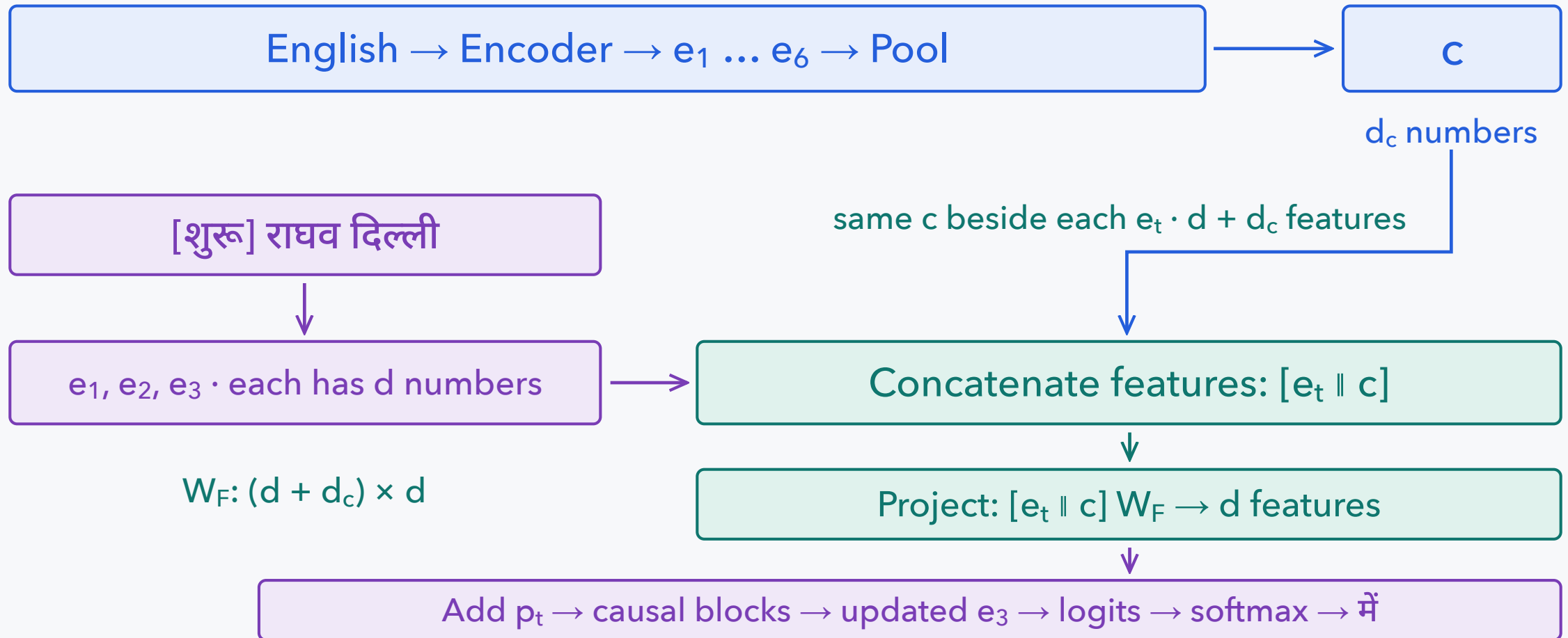
From target embeddings to the next Hindi token



Read the last target embedding; append मैं and repeat.

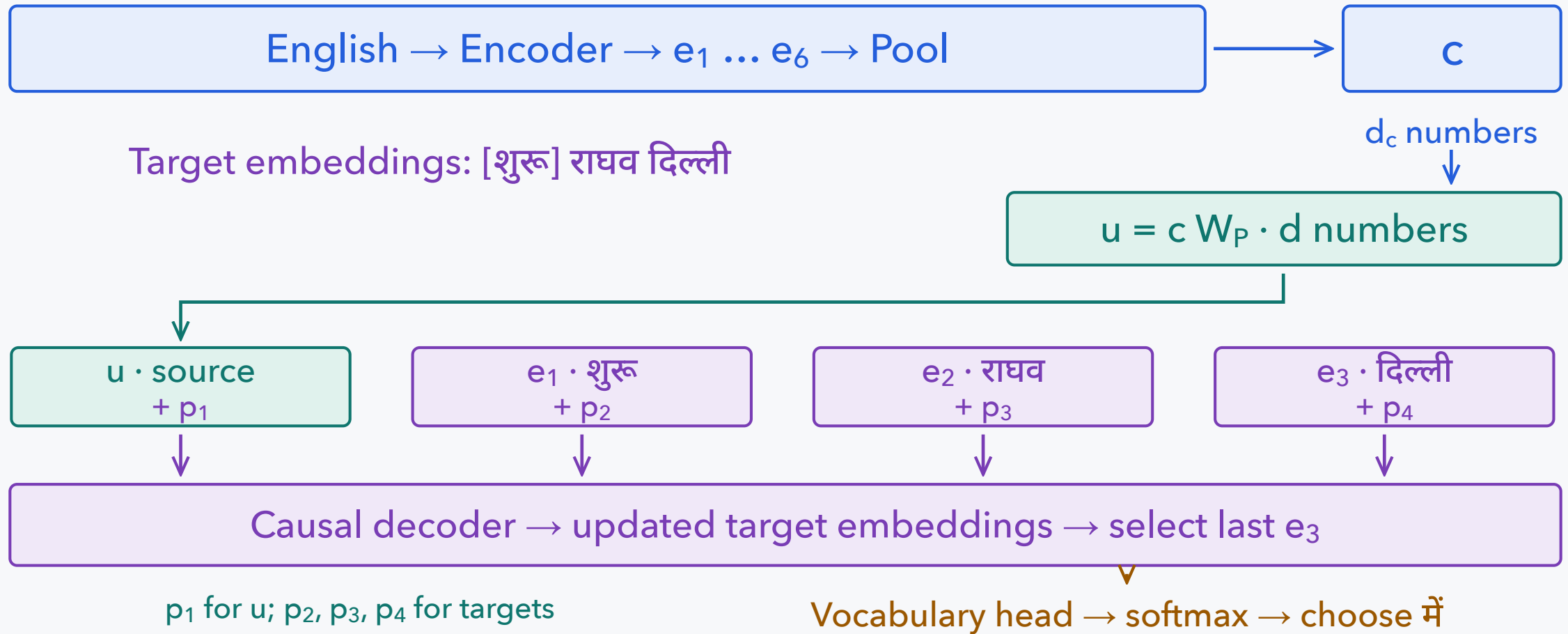
The same decoder and vocabulary head are reused at the next generation step.

Option 2: concatenate context as extra features



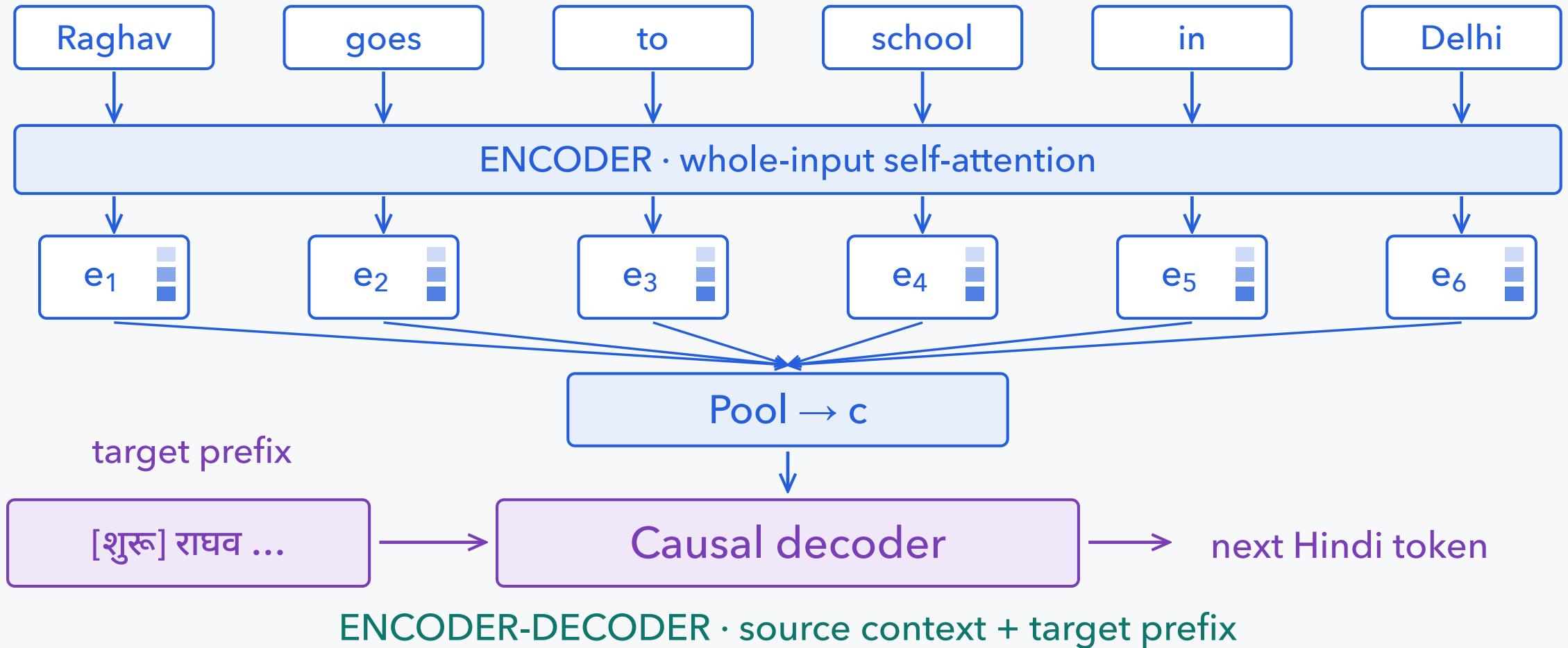
Concatenation increases the feature width; it does not add a token position.

Option 3: prepend context as an extra input position



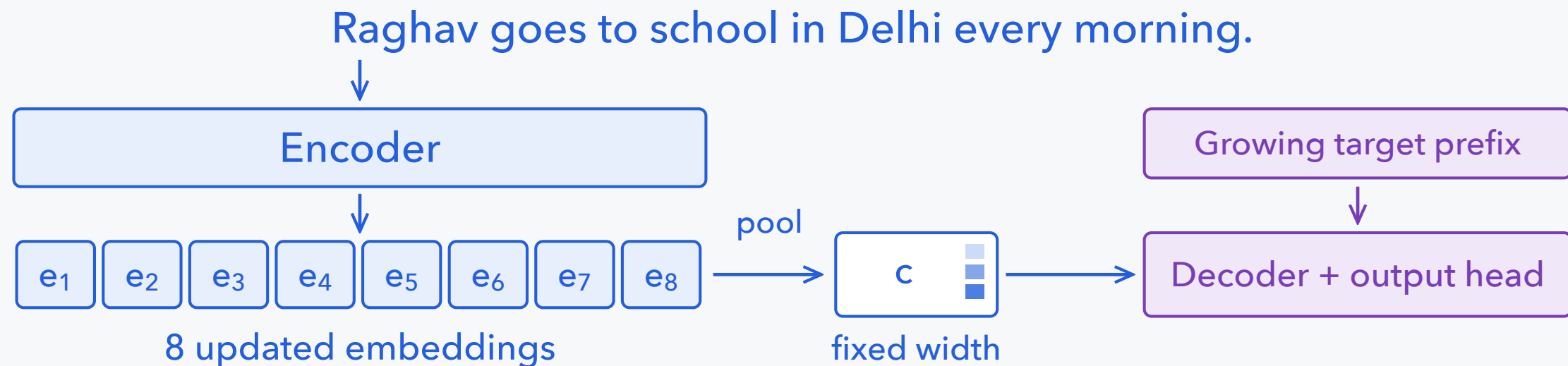
The target reads the source-prefix vector through ordinary causal self-attention.

An encoder-decoder generates a target from a source



Represent the source + generate the target.

The decoder cannot revisit the source



Different target steps need different source details

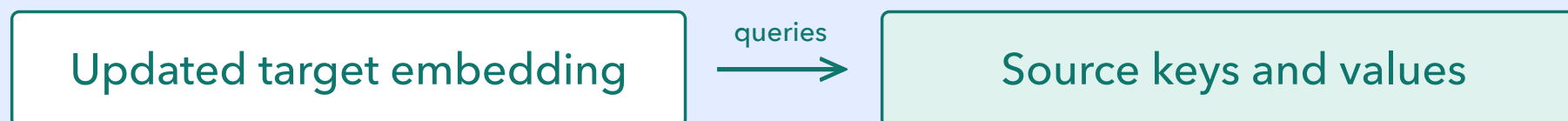


If c loses "Delhi", the decoder cannot look back to recover it.

The decoder relies on c for all source information.

Part IV · Cross-attention to the source

Retain one encoder output per source position.



Let each target position query the source directly.

01 · Generation

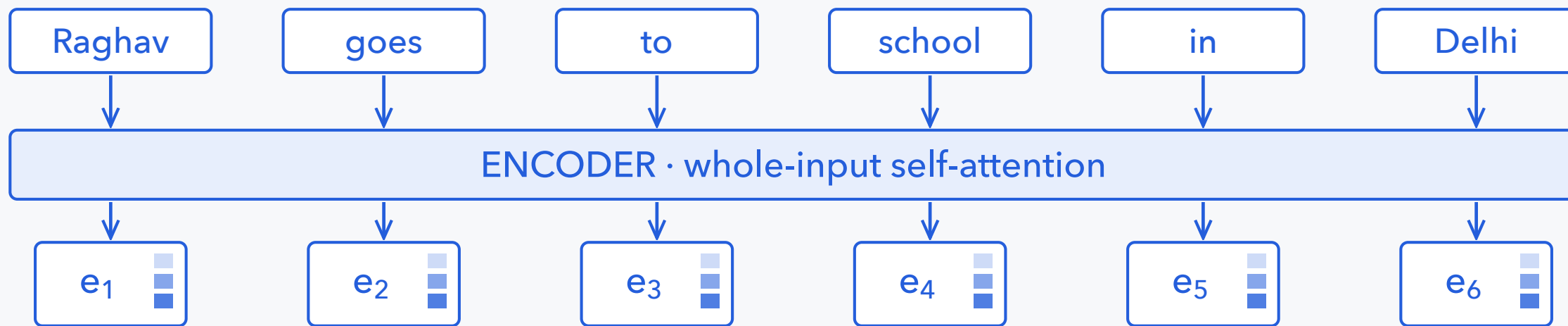
02 · Input labels

03 · Translation

04 · Cross-attention

05 · Model families

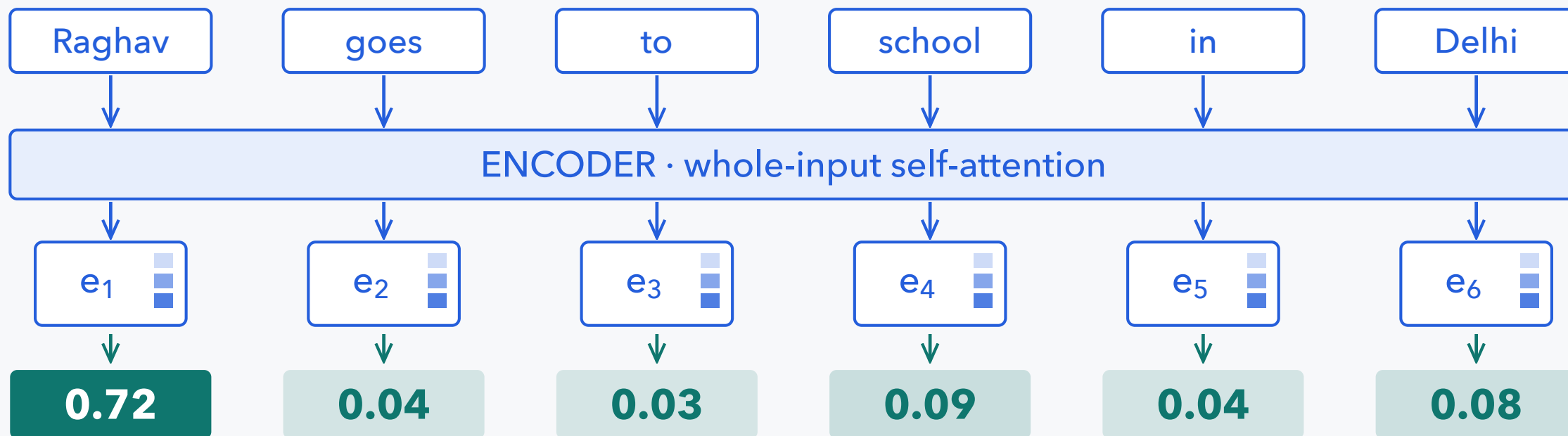
Retaining the source embeddings for the decoder



Keep one contextual vector per source position.

Keep $e_1 \dots e_6$ available to the decoder.

Source attention when predicting राघव



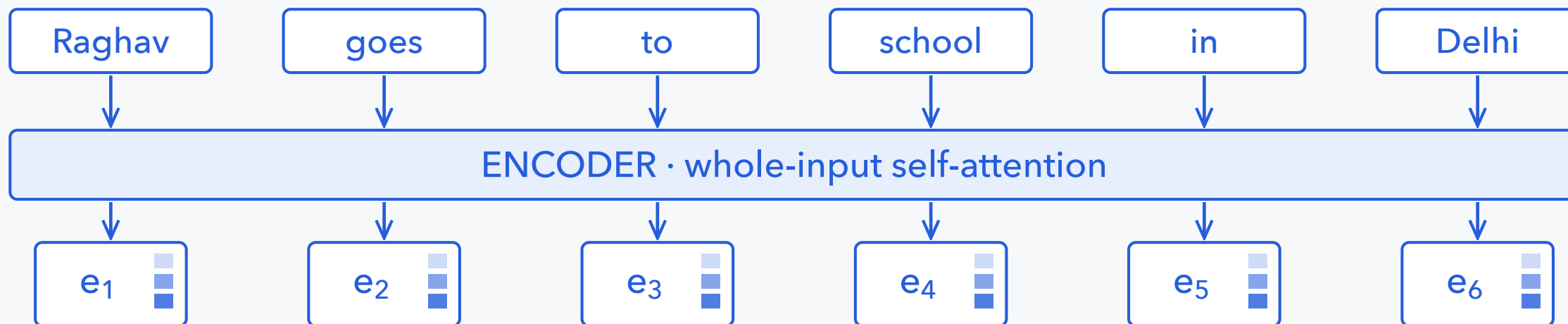
An illustrative lookup: put most weight on the English name Raghav.

Target prefix: [शुरू] → example next Hindi token: राघव

Illustrative attention weights; their sum is 1.

All six English embeddings are available; the target query can assign more weight to Raghav.

Step 1: encode the English sentence

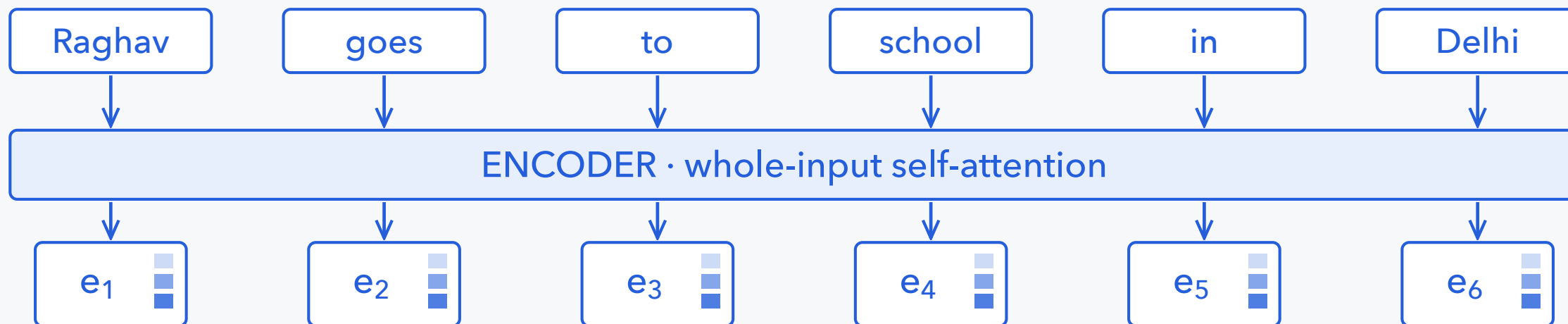


Six encoder outputs, one for each English position.

Keep these contextual embeddings available to the decoder.

Retain all six encoder outputs before starting the target decoder.

Step 2: initialise the target at [शुरू]



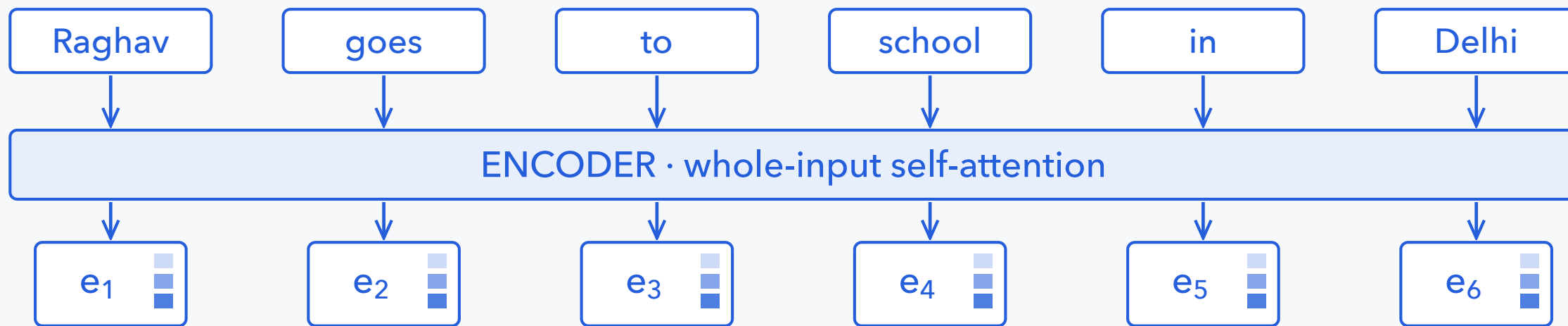
TARGET: begin with the special token [शुरू]



राघव has not been generated; the supplied target is only [शुरू].

| Look up the start-token embedding and add its position vector.

Step 3: update [शुरू] with target self-attention



TARGET: begin with the special token [शुरू]

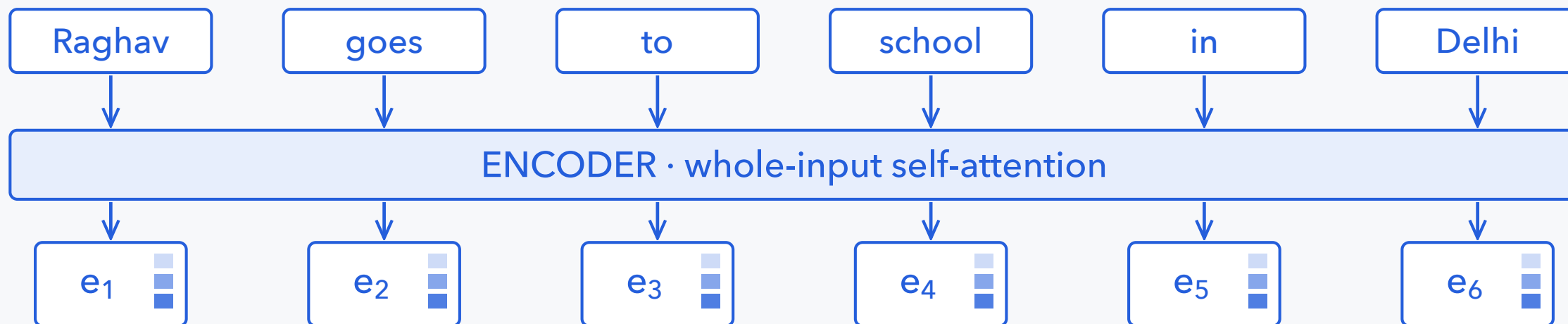


$$\text{Target } Q, K, V \rightarrow m_1 \rightarrow \Delta e_1 \rightarrow e_1 + \Delta e_1$$

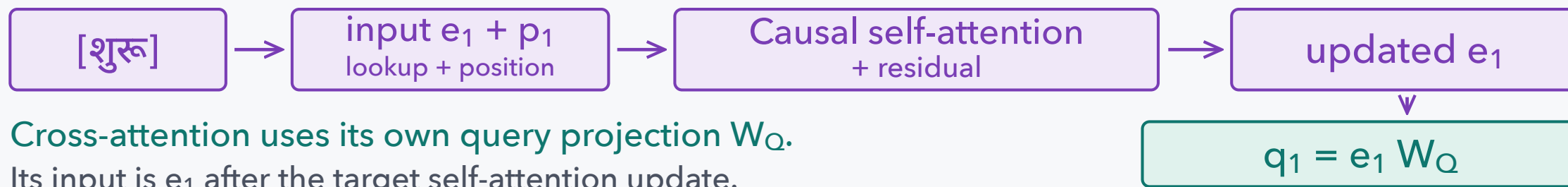
Only [शुरू] is available: self-attention reads this one target position.

Causal self-attention updates the target embedding before cross-attention.

Step 4: form a new query for cross-attention



TARGET: begin with the special token [शुरू]

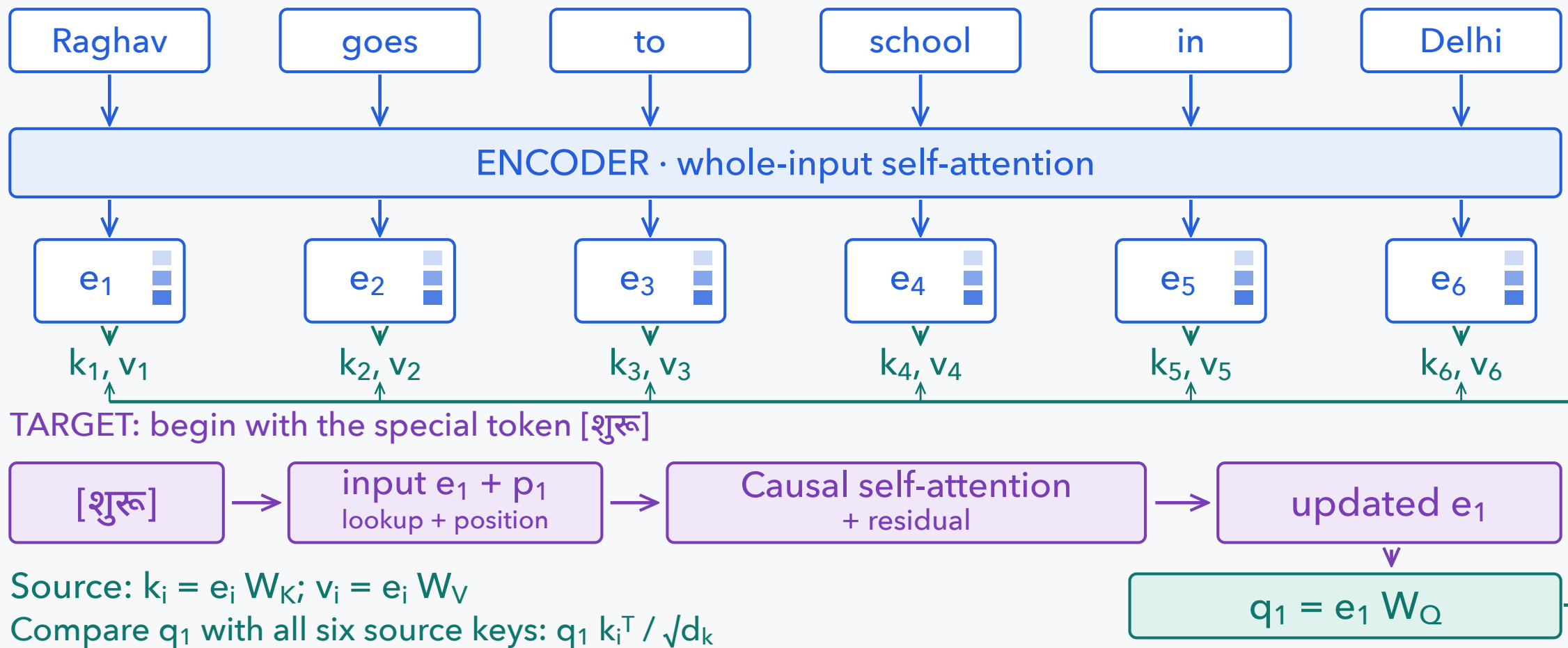


Cross-attention uses its own query projection W_Q .

Its input is e_1 after the target self-attention update.

Self-attention and cross-attention have separate learned query projections.

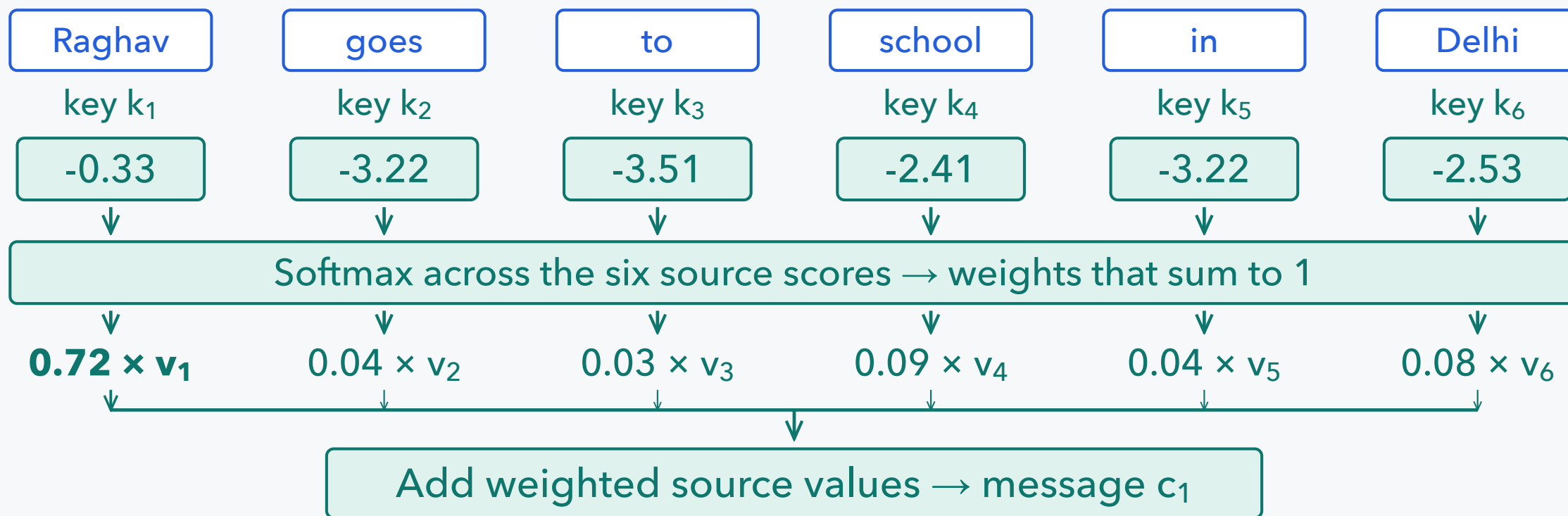
Step 5: match the target query to English keys



Cross-attention: Q from the updated target; K and V from encoder outputs.

Computing the weighted source message

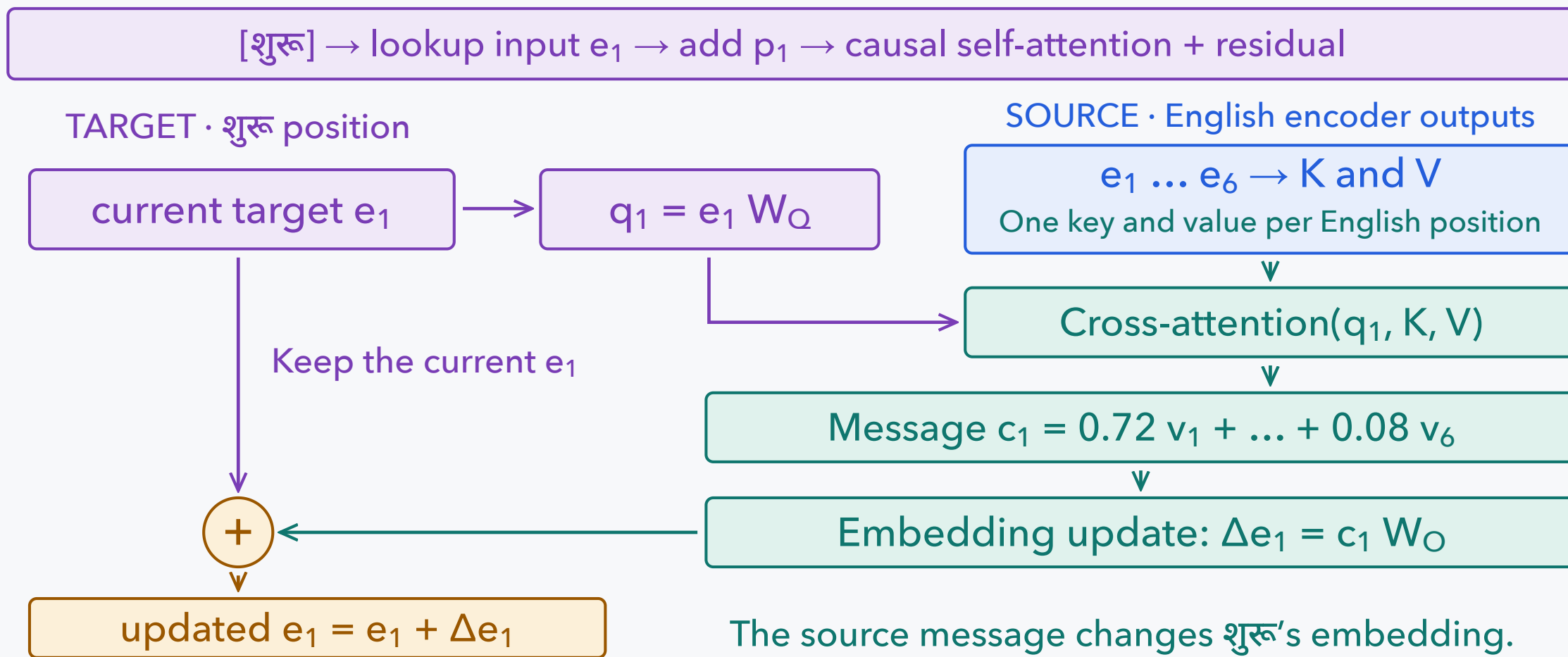
Match the target q_1 with each English key: $q_1 k_i^T / \sqrt{d_k}$



$c_1 W_O + \text{target } e_1 \rightarrow \text{MLP} + \text{residual} \rightarrow \text{vocabulary head} \rightarrow \text{राघव}$

Keys determine the weights; values supply the information sent to the target.

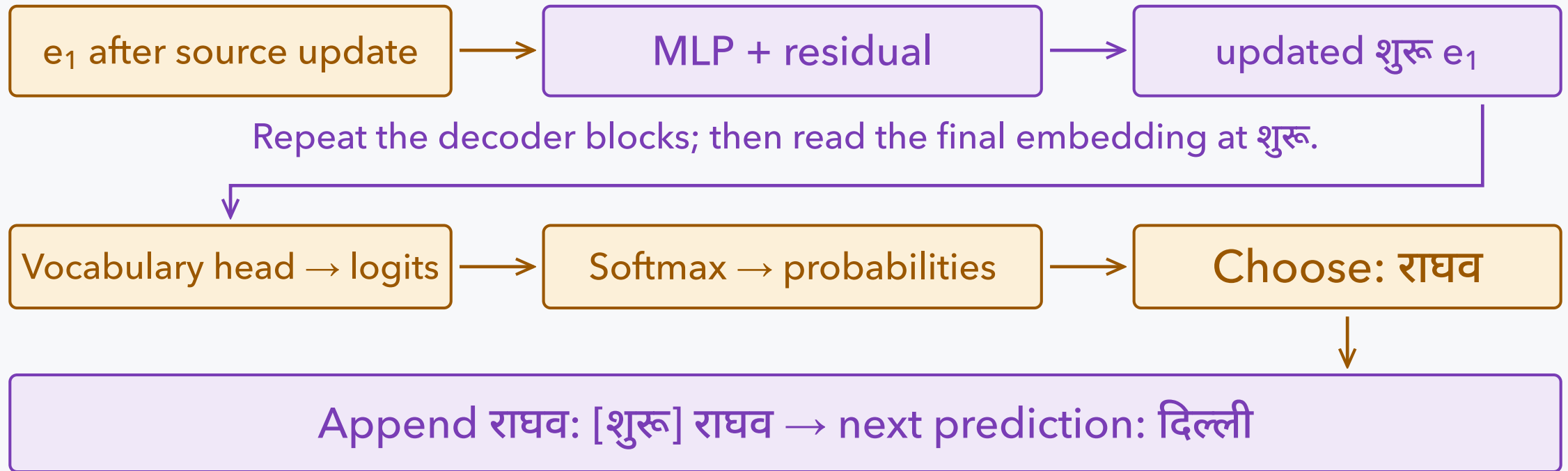
The source message updates शुरु's embedding



Keep the current target embedding, then add the update retrieved from the source.

Read the updated शुरू embedding to predict राघव

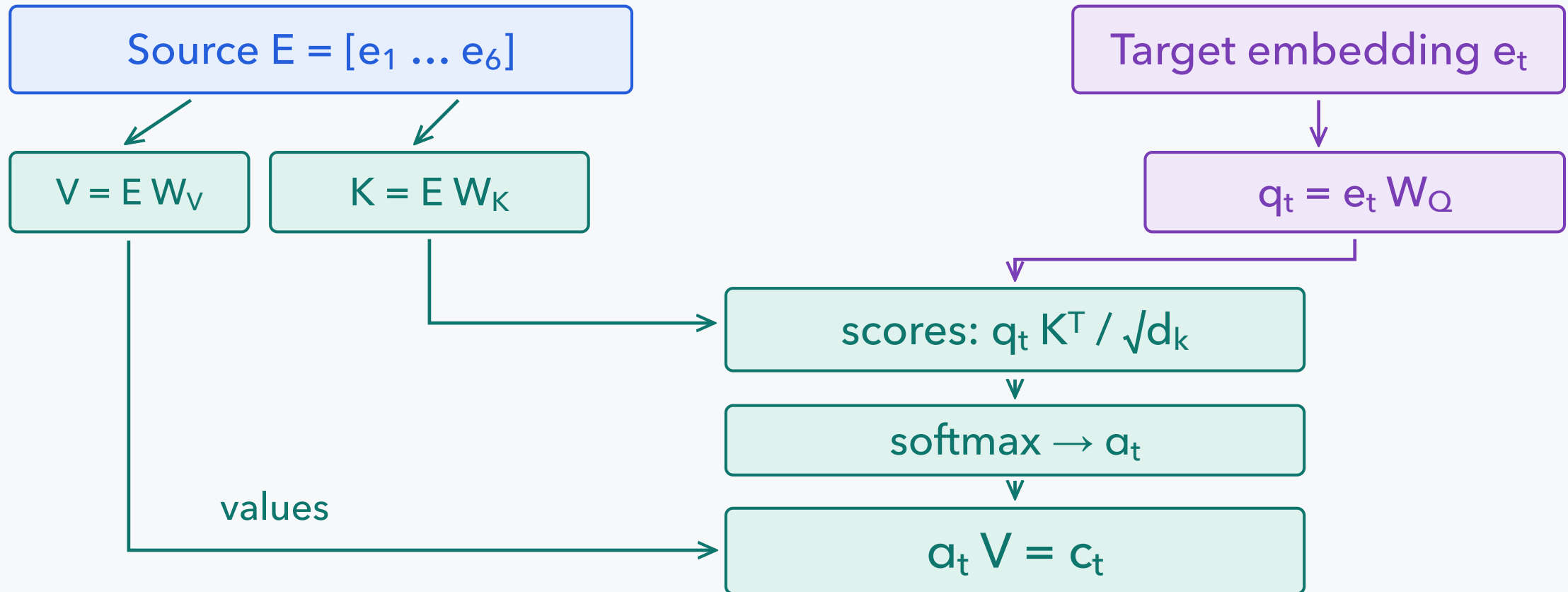
The शुरू embedding after cross-attention.



e_1 still belongs to शुरू. The new token राघव gets input embedding e_2 .

शुरू keeps its position; its final embedding is used to predict the next token.

The roles of keys and values



Query-key scores determine the weights used to combine the values.

The cross-attention calculation

One target query, all source keys and values

$$q_t = e_t W_Q$$

$$k_i = e_i W_K$$

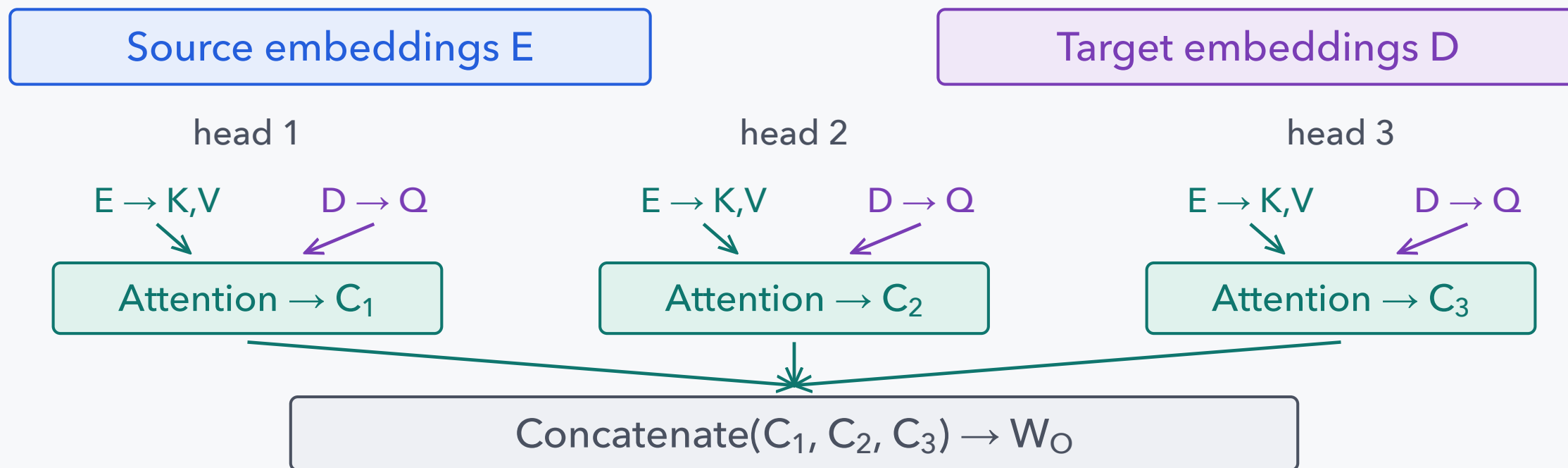
$$v_i = e_i W_V$$

$$a_{ti} = \text{softmax over } i (q_t k_i^T / \sqrt{d_k})$$

$$c_t = \sum_i a_{ti} v_i$$

| c_t is the cross-attention message, just as m_i was the self-attention message.

Multi-head cross-attention uses the same mechanism



$$Q_j = D W_Q^{(j)} \cdot K_j = E W_K^{(j)} \cdot V_j = E W_V^{(j)}$$

$$C_j = \text{softmax}(Q_j K_j^T / \sqrt{d_k}) V_j$$

Each head uses target queries and source keys and values.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_1 at [शुरू]

Keys and values: supplied Hindi positions

[शुरू]

1.00

$$m_1 = \sum \alpha_i v_i \cdot \text{Hindi values}$$

$\Delta e_1 = m_1 W_O$; add to current e_1
Updated e_1 after self-attention

2 · English cross-attention

Same six English encoder outputs

Source keys K and values V

Raghav

goes

to

school

in

Delhi

$q_1 = \text{updated } e_1 W_Q \text{ (cross-attention)}$
 $q_1 K^T / \sqrt{d_k} \rightarrow \text{softmax over English}$

0.72

0.04

0.03

0.09

0.04

0.08

$$c_1 = \sum \alpha_i v_i \cdot \text{English values}$$

Cross update: $\Delta e_1 = c_1 W_O \rightarrow \text{add to } e_1 \rightarrow \text{MLP + residual; finish blocks}$

Step 1: vocabulary head $\rightarrow$ softmax $\rightarrow$ राघव · append to Hindi prefix

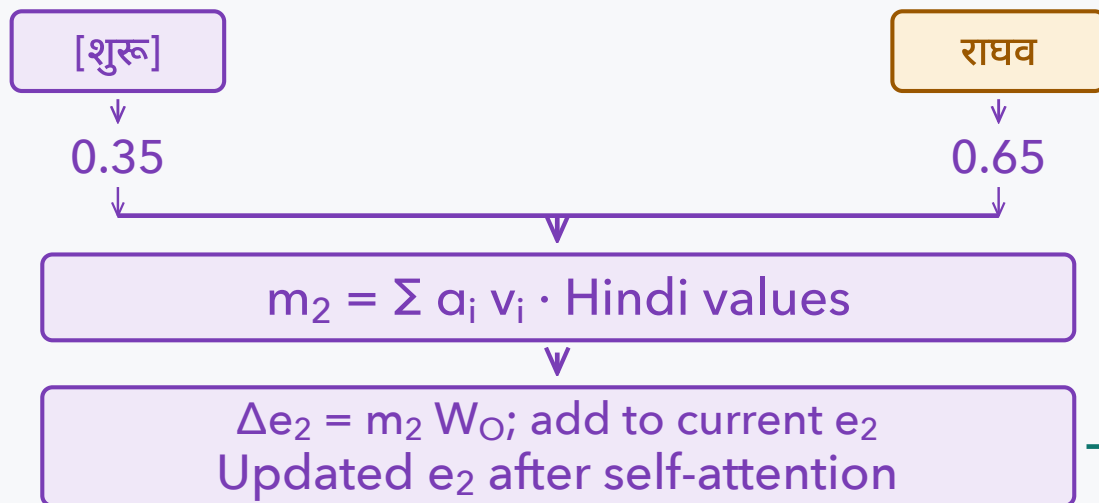
Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_2 at राघव

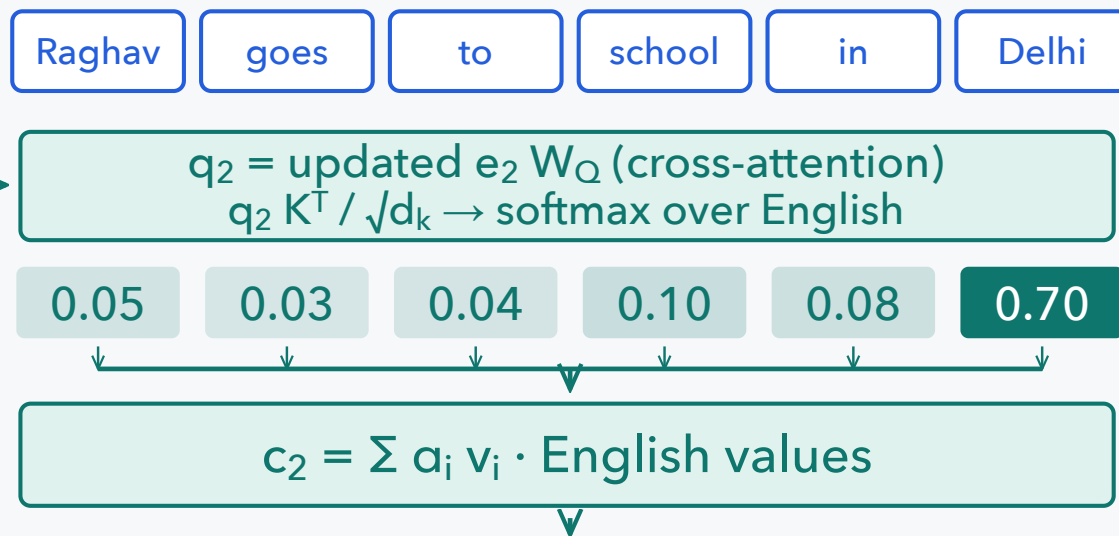
Keys and values: supplied Hindi positions



2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



Cross update: $\Delta e_2 = c_2 W_O \rightarrow \text{add to } e_2 \rightarrow \text{MLP + residual; finish blocks}$

Step 2: vocabulary head $\rightarrow$ softmax $\rightarrow$ दिल्ली · append to Hindi prefix

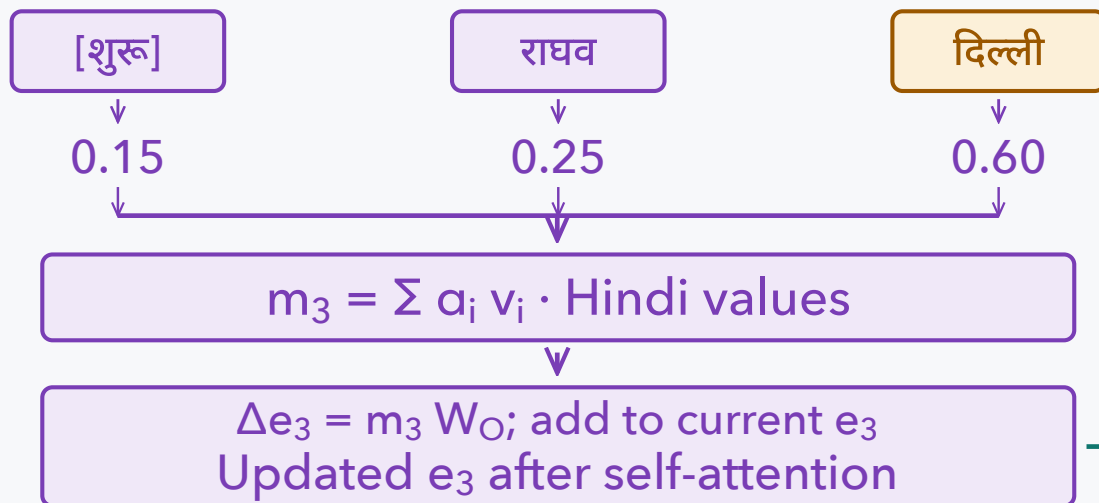
Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_3 at दिल्ली

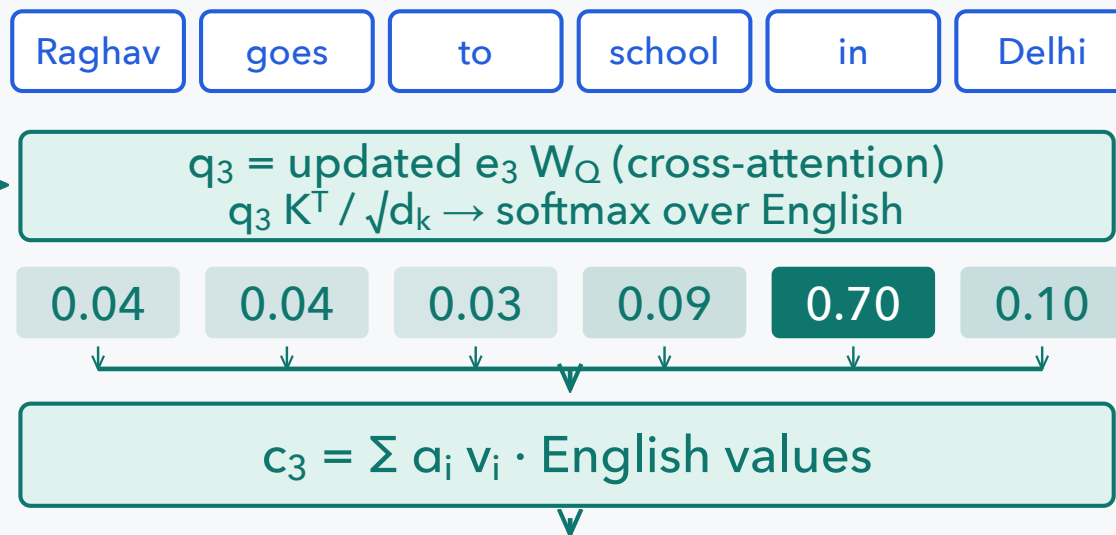
Keys and values: supplied Hindi positions



2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



Cross update: $\Delta e_3 = c_3 W_O \rightarrow \text{add to } e_3 \rightarrow \text{MLP + residual; finish blocks}$

Step 3: vocabulary head $\rightarrow$ softmax $\rightarrow$ मैं · append to Hindi prefix

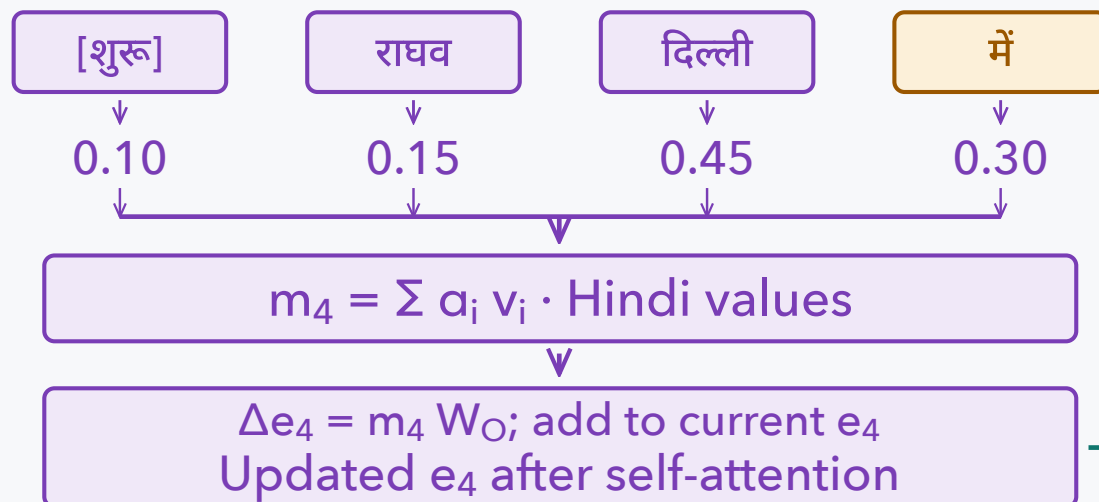
Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_4 at में

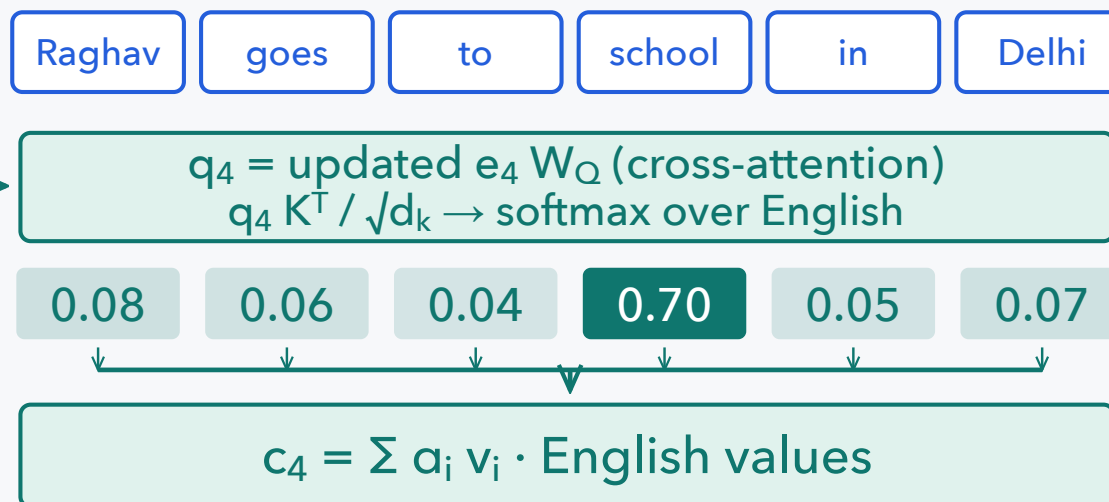
Keys and values: supplied Hindi positions



2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



Cross update: $\Delta e_4 = c_4 W_O \rightarrow \text{add to } e_4 \rightarrow \text{MLP + residual; finish blocks}$

Step 4: vocabulary head $\rightarrow$ softmax $\rightarrow$ स्कूल · append to Hindi prefix

Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_5 at स्कूल

Keys and values: supplied Hindi positions



$$m_5 = \sum \alpha_i v_i \cdot \text{Hindi values}$$

$\Delta e_5 = m_5 W_O$; add to current e_5
Updated e_5 after self-attention

2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



$q_5 = \text{updated } e_5 W_Q \text{ (cross-attention)}$
 $q_5 K^T / \sqrt{d_k} \rightarrow \text{softmax over English}$



$$c_5 = \sum \alpha_i v_i \cdot \text{English values}$$

Cross update: $\Delta e_5 = c_5 W_O \rightarrow \text{add to } e_5 \rightarrow \text{MLP + residual; finish blocks}$

Step 5: vocabulary head $\rightarrow$ softmax $\rightarrow$ जाता · append to Hindi prefix

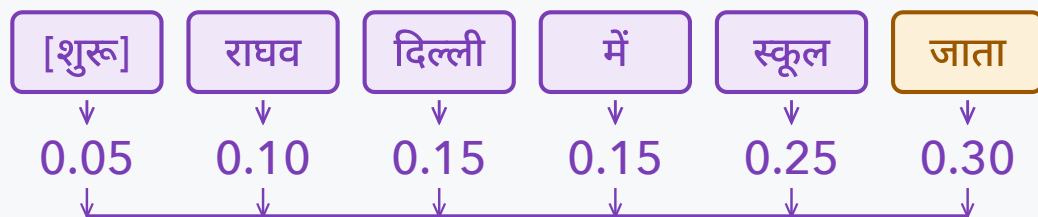
Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_6 at जाता

Keys and values: supplied Hindi positions



$$m_6 = \sum \alpha_i v_i \cdot \text{Hindi values}$$

$\Delta e_6 = m_6 W_O$; add to current e_6
Updated e_6 after self-attention

2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



$q_6 = \text{updated } e_6 W_Q \text{ (cross-attention)}$
 $q_6 K^T / \sqrt{d_k} \rightarrow \text{softmax over English}$



$$c_6 = \sum \alpha_i v_i \cdot \text{English values}$$

Cross update: $\Delta e_6 = c_6 W_O \rightarrow \text{add to } e_6 \rightarrow \text{MLP + residual; finish blocks}$

Step 6: vocabulary head $\rightarrow$ softmax $\rightarrow$ है · append to Hindi prefix

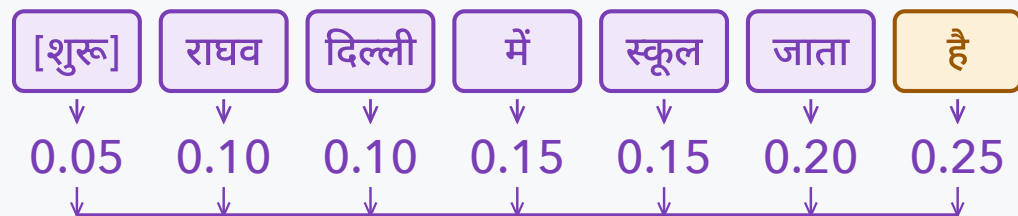
Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Hindi generation: self-attention, then cross-attention

1 · Hindi self-attention

Self query from e_7 at है

Keys and values: supplied Hindi positions



$$m_7 = \sum \alpha_i v_i \cdot \text{Hindi values}$$

$\Delta e_7 = m_7 W_O$; add to current e_7
Updated e_7 after self-attention

2 · English cross-attention

Same six English encoder outputs

Source keys K and values V



$q_7 = \text{updated } e_7 W_Q \text{ (cross-attention)}$
 $q_7 K^T / \sqrt{d_k} \rightarrow \text{softmax over English}$

0.16	0.18	0.16	0.17	0.16	0.17
------	------	------	------	------	------

Arrows from the first five values (0.16 to 0.16) point to the next step.

$$c_7 = \sum \alpha_i v_i \cdot \text{English values}$$

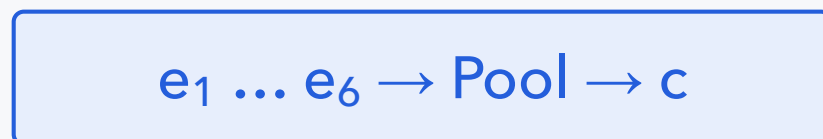
Cross update: $\Delta e_7 = c_7 W_O \rightarrow \text{add to } e_7 \rightarrow \text{MLP + residual; finish blocks}$

Step 7: vocabulary head $\rightarrow$ softmax $\rightarrow$ [समाप्त] · stop

Hindi context grows; English context stays fixed. Both sets of weights are illustrative.

Fixed context and query-dependent context

ONE FIXED SOURCE VECTOR

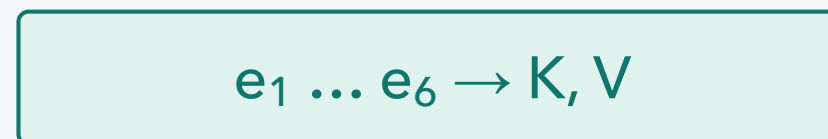


$c \longrightarrow \text{predict } y_1$

$c \longrightarrow \text{predict } y_2$

$c \longrightarrow \text{predict } y_3$

SOURCE STATES RETAINED



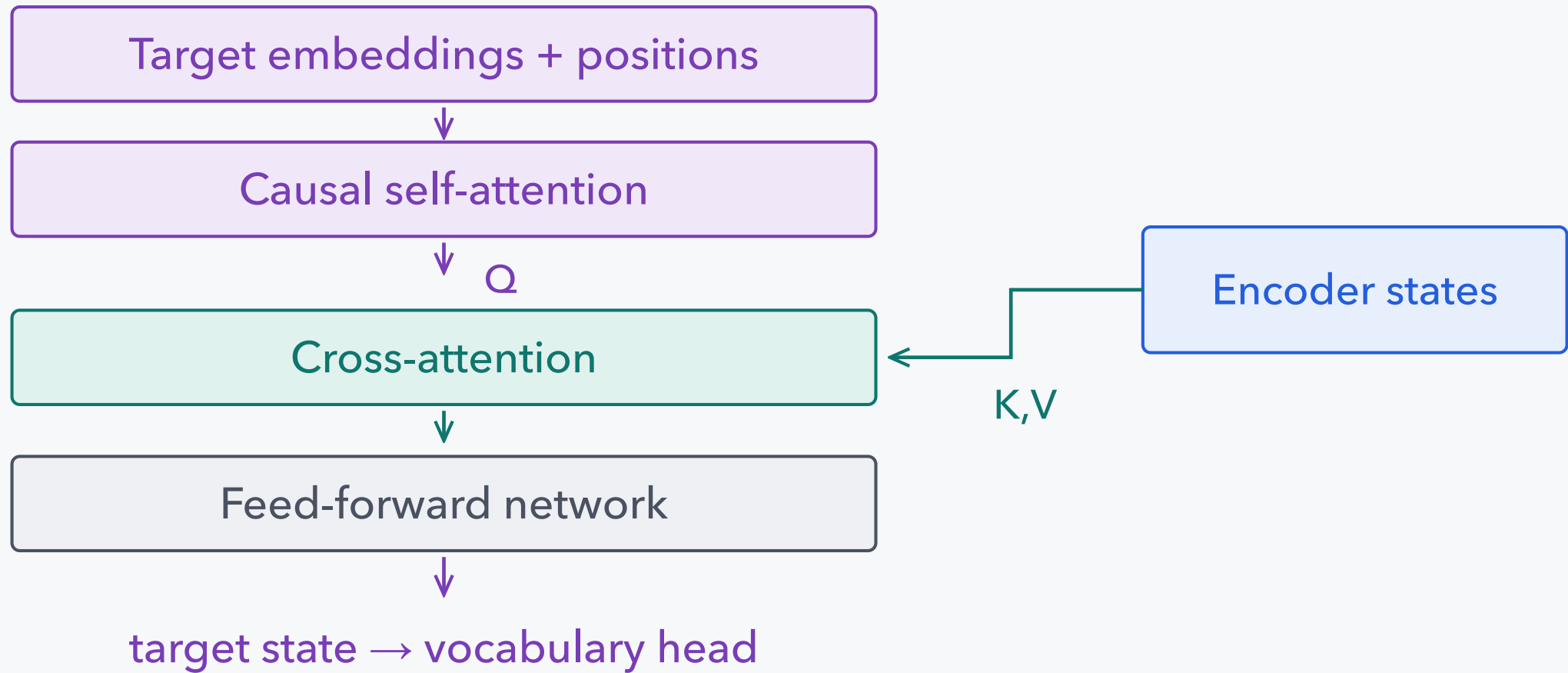
$q_1 \longrightarrow \text{lookup} \longrightarrow c_1$

$q_2 \longrightarrow \text{lookup} \longrightarrow c_2$

$q_3 \longrightarrow \text{lookup} \longrightarrow c_3$

Source states stay fixed; the query-dependent message changes.

One decoder block, two attention operations



Residuals + normalization omitted for clarity.

Step 1: update the target embedding with self-attention

Hindi prefix: [शुरू] राघव

Input e_1, e_2 + positions

1 · Hindi causal self-attention
राघव reads [शुरू] and itself · target Q, K, V

$m_2 \rightarrow \Delta e_2 \rightarrow$ add to the current e_2

e_2 after the Hindi update

Same target position: राघव.

Its embedding now reflects the prefix.

English $\rightarrow$ encoder outputs E

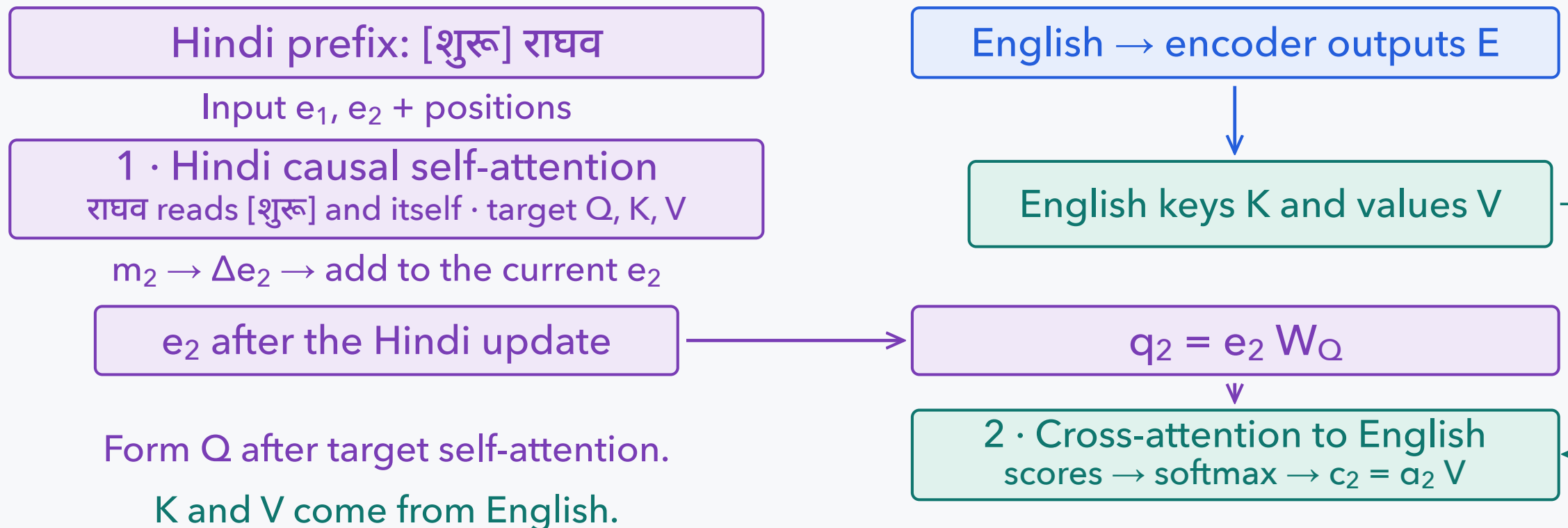


English keys K and values V

No next token is predicted here. We are updating the supplied embeddings.

Self-attention: target Q, target K, target V. Update the existing target embeddings.

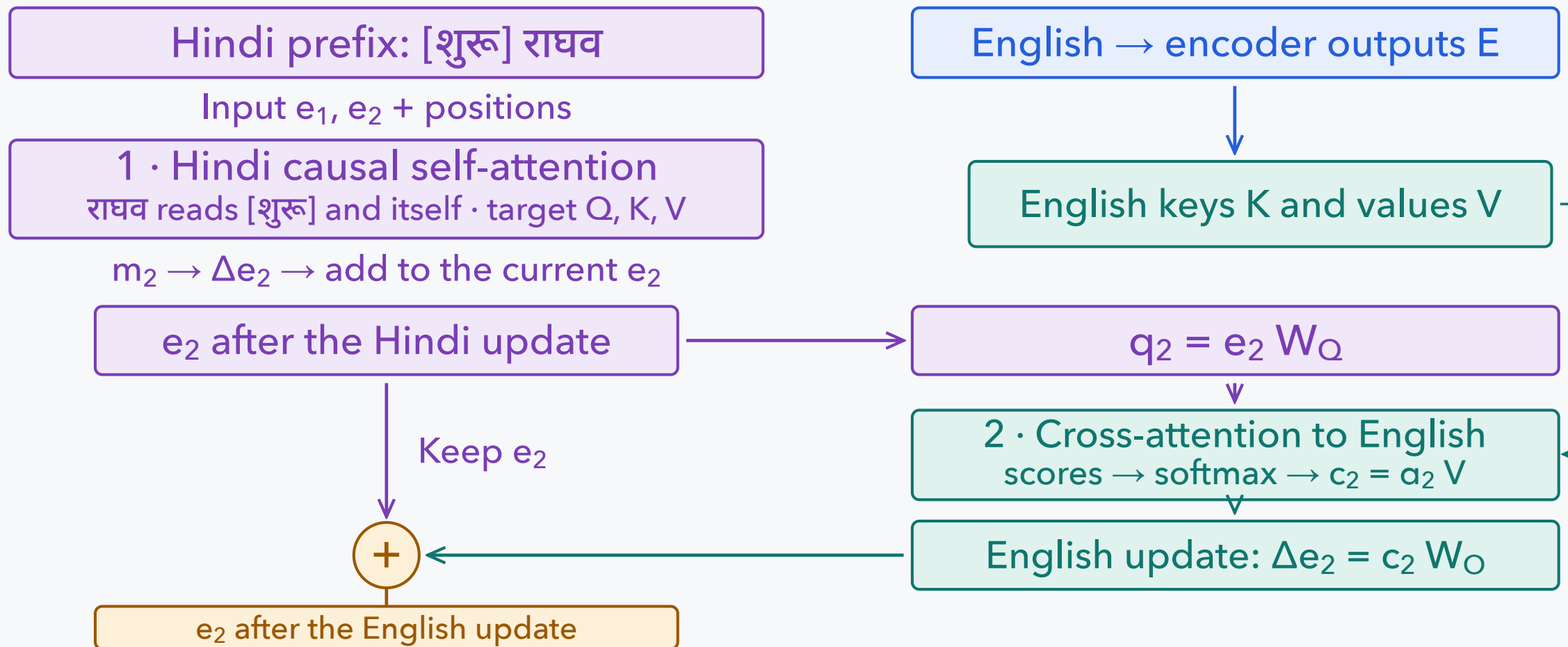
Step 2: form a query for cross-attention



c_2 is the retrieved source message; next, add its projected update to e_2 .

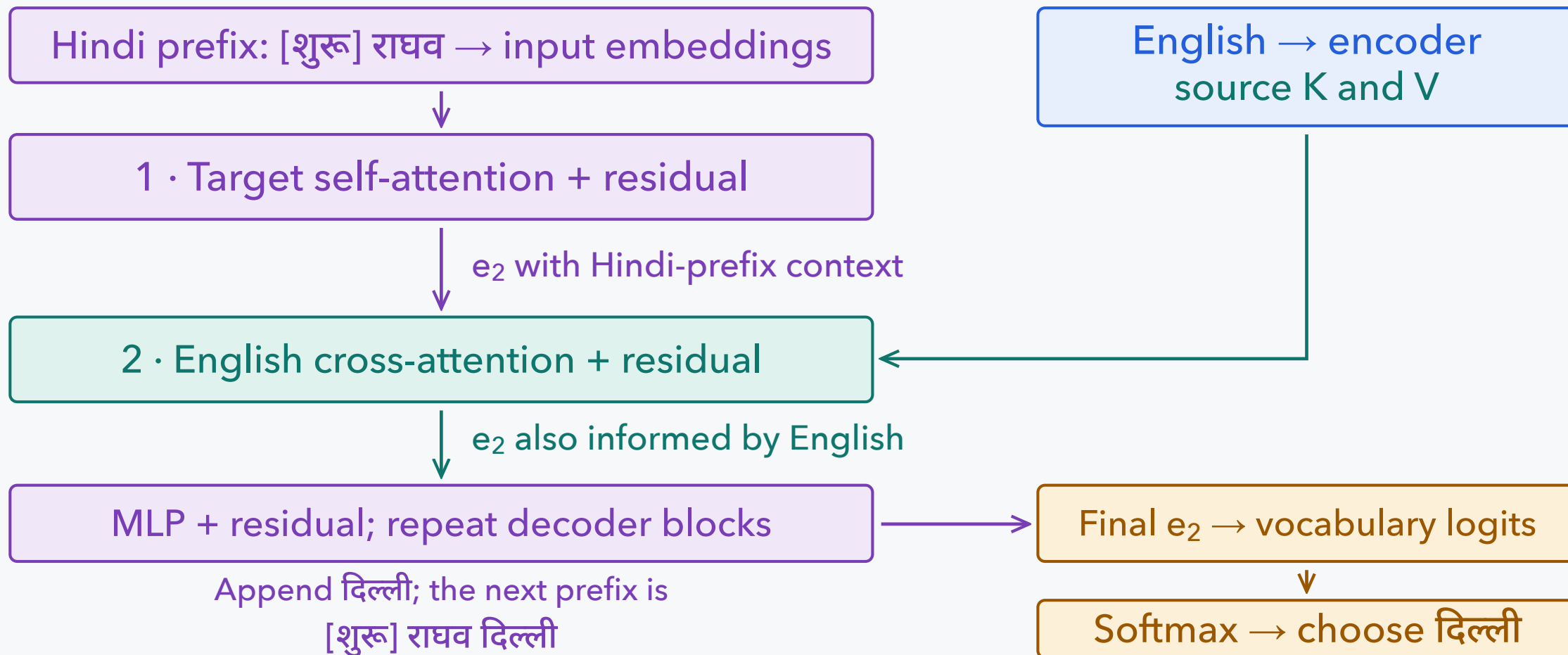
Cross-attention: Q from the updated target embedding; K and V from English.

Step 3: add the projected source message



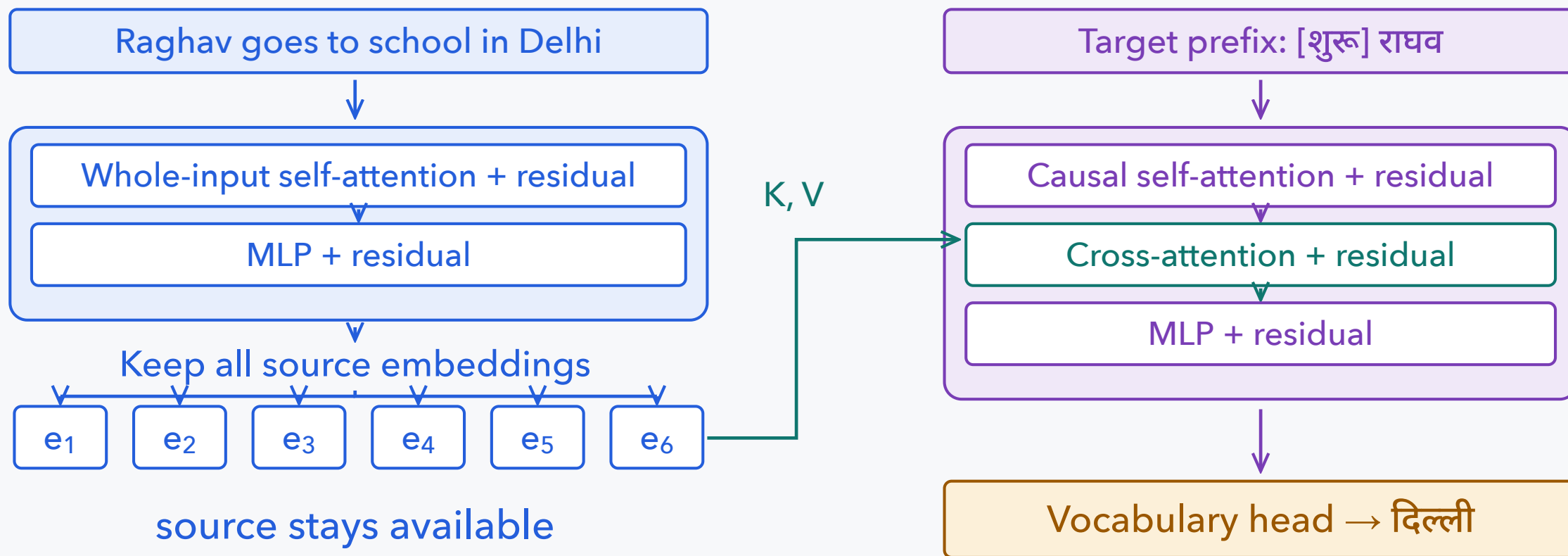
Two successive updates to e_2 : first from Hindi, then from English.

Step 4: predict the next token



Hindi self-attention → English cross-attention → MLP → final vocabulary prediction.

Encoder-decoder: generation conditioned on a source



Each target step uses the prefix and the source embeddings.

Two sequences: the source stays available while the target grows.

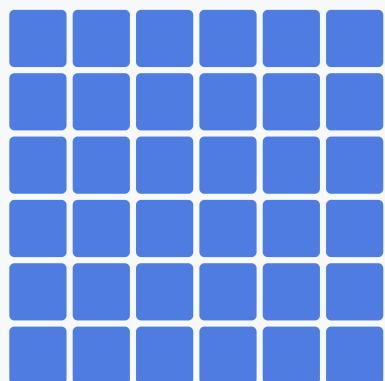
Attention patterns and their applications

English example: Raghav goes to school in Delhi

Encoder self-attention

Classify the topic: EDUCATION

English → English



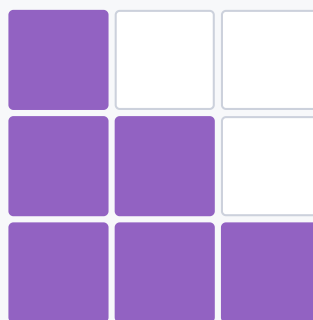
Q, K, V: English

$p(\text{label} \mid \text{complete input})$

Decoder self-attention

Continue: Raghav goes to ...

Prefix → prefix



Q, K, V: prefix

$p(\text{school} \mid \text{Raghav goes to})$

Cross-attention

Translate: [शुरू] राघव ...

Hindi → English



Q: Hindi • K, V: English

$p(\text{दिल्ली} \mid \text{English, Hindi prefix})$

Filled cells show allowed connections. The task probability comes from the final head.

Encoder attention for sentence classification

Input x: Raghav goes to school in Delhi

Columns: English keys / values

	Raghav	goes	to	school	in	Delhi
Raghav						
goes						
to						
school						
in						
Delhi						

Rows: English queries • all six words are visible

Encoder → 6 updated embeddings



Mean pool → one sentence vector



Topic head → logits → softmax

$$p(y = \text{EDUCATION} \mid x)$$

The encoder keeps all positions. Pooling gives the summary used by this task.

Example task: classify the sentence's topic using all supplied words.

Causal attention for next-token prediction

Supplied prefix: Raghav goes to → next token: school

Columns: supplied prefix keys / values

	Raghav	goes	to
Raghav			
goes			
to			

The "to" row can read Raghav, goes and to.

Causal attention updates embeddings



Finish blocks → read last e_3



Vocabulary logits → softmax

$p(\text{school} \mid \text{Raghav goes to})$

Previous words are available; future words cannot be read.

The current and previous supplied words are visible; future words are not.

Cross-attention between Hindi and English

English x: Raghav goes to school in Delhi

Columns: English keys / values

	Raghav	goes	to	school	in	Delhi
[शुरू]						
राघव						

Rows: Hindi queries after causal self-attention

Hindi prefix: [शुरू] राघव



Q: Hindi · K and V: English



Source message → target update

Both Hindi positions may read all six English positions.

Translation task: $p(\text{दिल्ली} \mid \text{English x, Hindi prefix [शुरू] राघव})$

Rows are Hindi target positions; columns are English source positions.

Part V · Comparing model architectures

Compare the architectures used for the example tasks.



Compare attention patterns and task heads.

01 · Generation

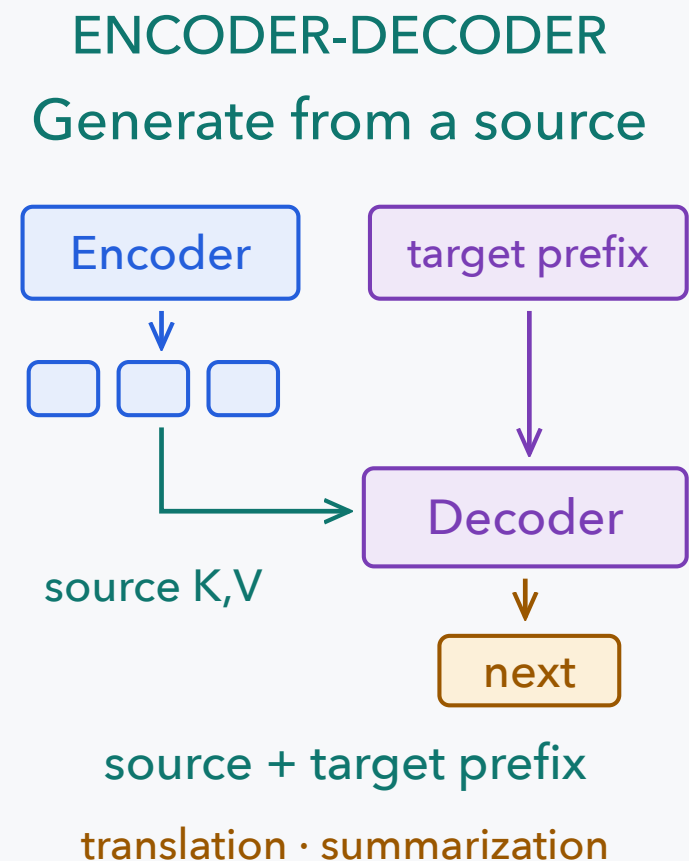
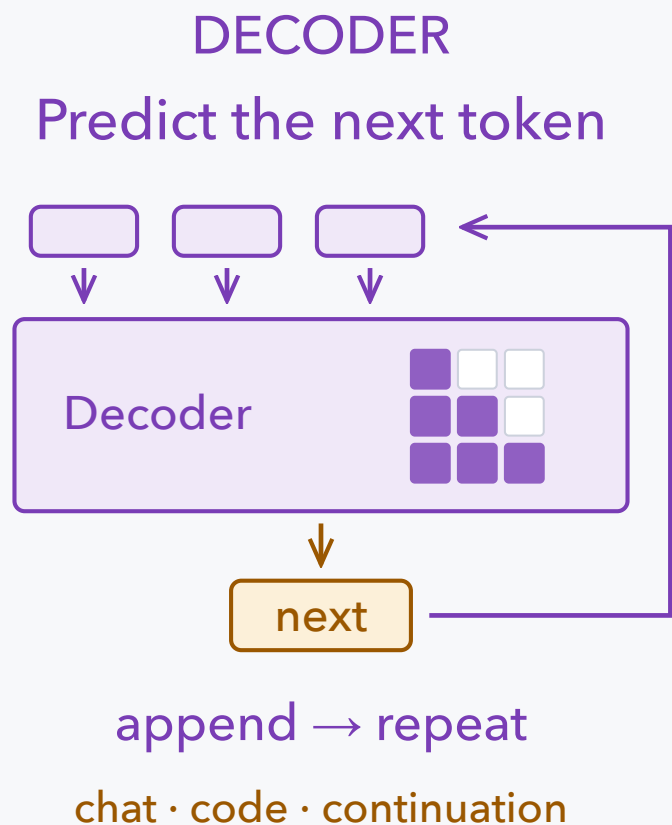
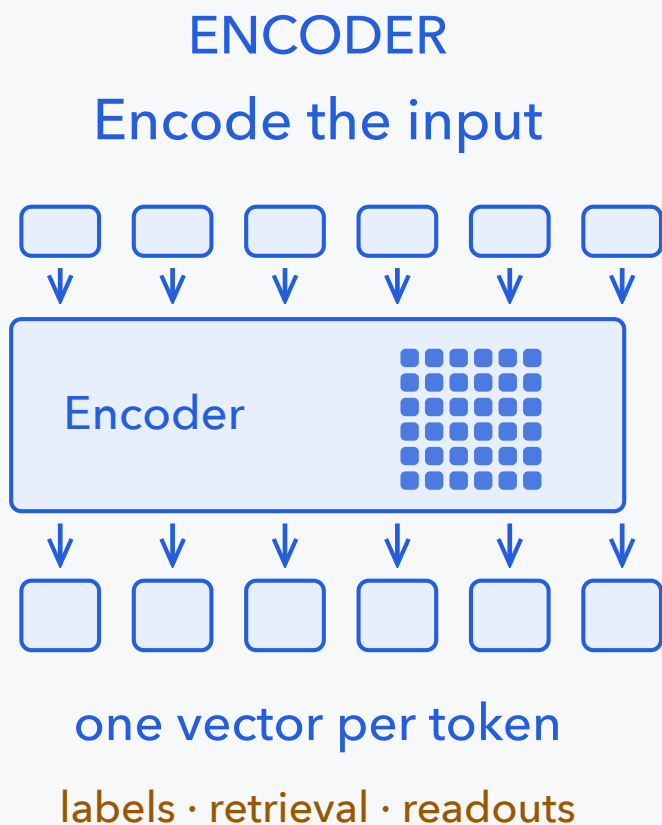
02 · Input labels

03 · Translation

04 · Cross-attention

05 · Model families

Encoder, decoder-only and encoder-decoder models



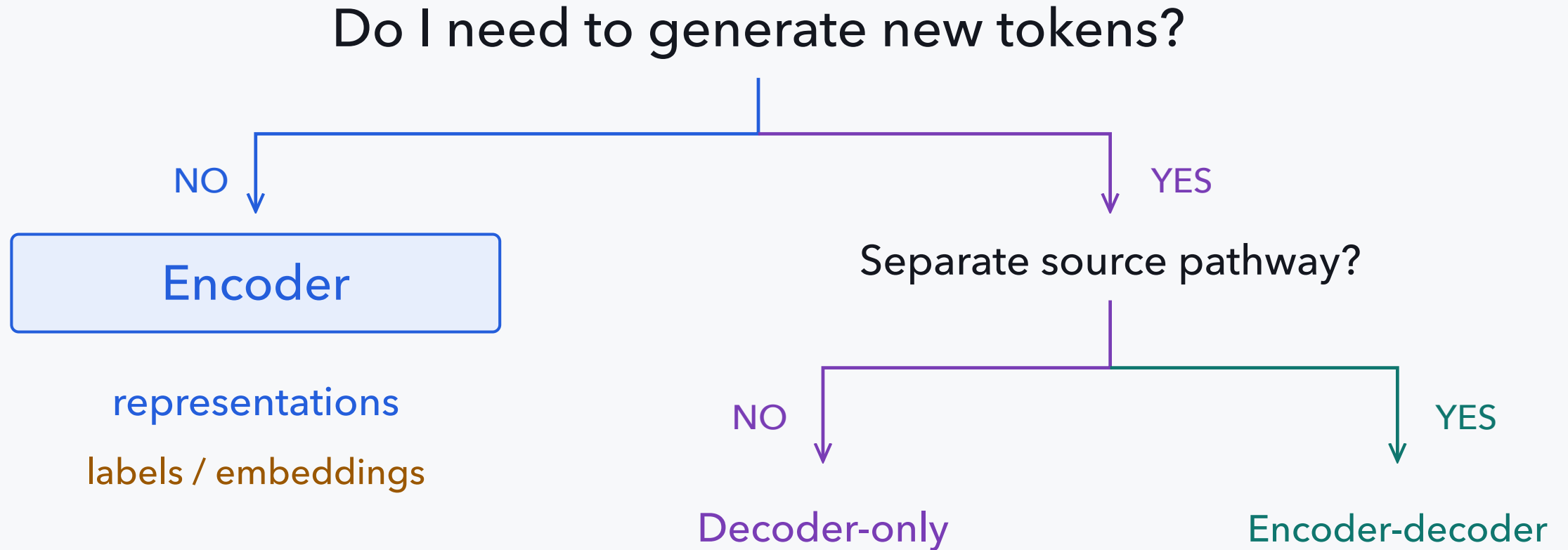
Compare the input, attention connections and output of each architecture.

Choosing an architecture for a task

One label for the input	Encoder + readout	sentiment
One label per token	Encoder + shared head	NER
Continue a prefix	Decoder-only	text generation
Turn source X into target Y	Encoder-decoder	English → Hindi

These are common choices; other architectures can perform the same tasks.

Choosing an architecture



| The output type and source pathway guide the choice of architecture.

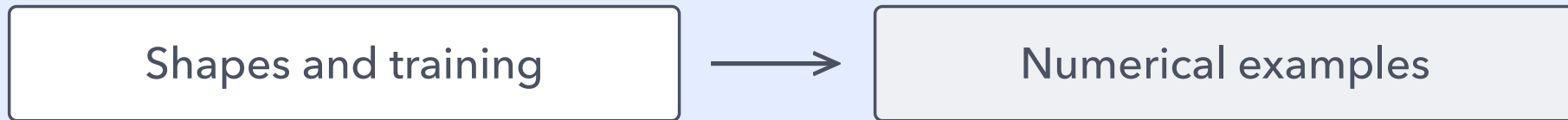
Review: attention, representations and training

- 01 WHICH POSITIONS CAN ATTEND TO EACH OTHER?
- 02 WHICH EMBEDDINGS DOES THE TASK USE?
- 03 WHAT IS THE TRAINING TARGET?

Next lecture: what if our tokens are not words?

Appendix: worked examples

Tensor shapes, training and a numerical attention example.



Optional details for discussion and practice.

Tensor shapes in cross-attention

$E \in \mathbb{R}^{n \times d}$ · source states

$D \in \mathbb{R}^{m \times d}$ · target states

d = model width

$$K \in \mathbb{R}^{n \times d_k}$$

$$V \in \mathbb{R}^{n \times d_v}$$

$$Q \in \mathbb{R}^{m \times d_k}$$

$$A = \text{softmax}(QK^T / \sqrt{d_k}) \in \mathbb{R}^{m \times n}$$

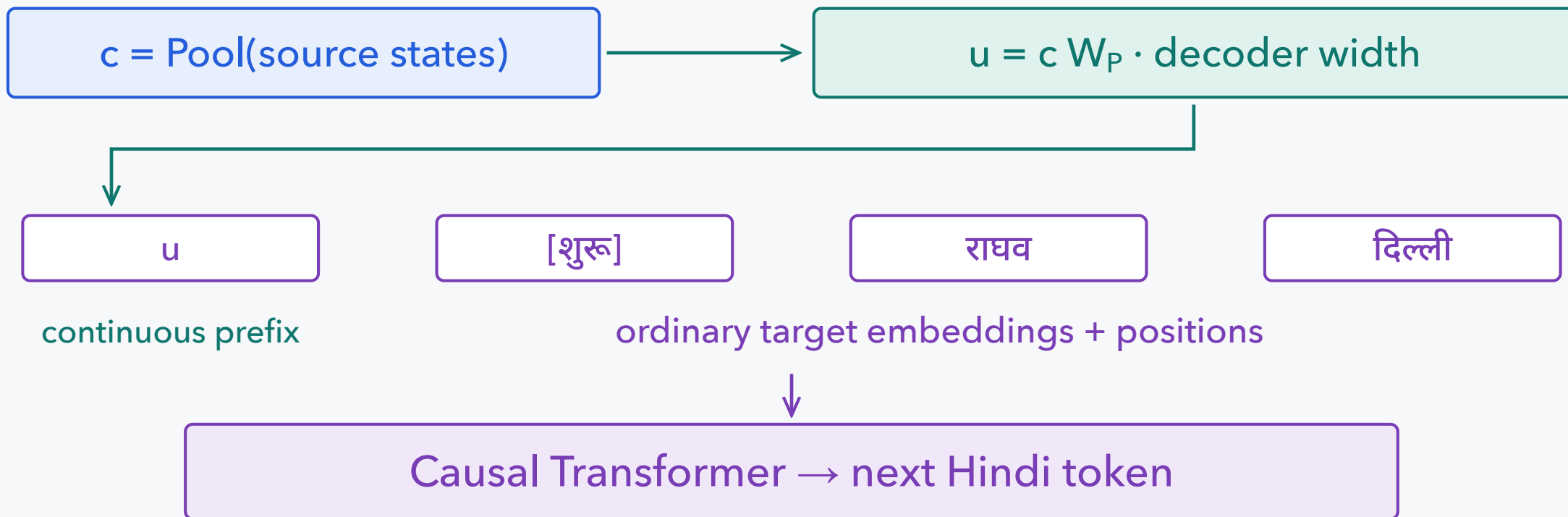


$$C = AV \in \mathbb{R}^{m \times d_v}$$

Example: $m = 4, n = 6 \rightarrow A$ is 4×6

| Rows are target queries; columns are source positions.

Conditioning with a continuous source prefix



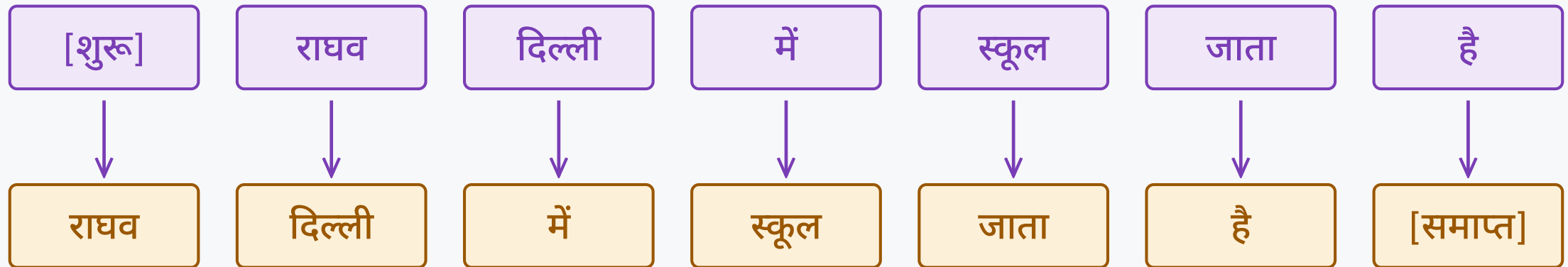
[CONTEXT] denotes an injected vector, not a new vocabulary word.

| The target can read the source prefix through causal self-attention.

Next-token prediction during training

Source: Raghav goes to school in Delhi.

Decoder input



Training target

$$L = -\sum_t \log p(y_t \mid y_{<t}, x)$$

Shift the target sequence by one position.

Training versus generation

TRAINING

correct previous target token



predict next token

Known target prefixes supplied

GENERATION

previous model prediction



predict next token

Append prediction; repeat

| The next-token objective is the same; prefix construction differs.

Attention head $\neq$ classification head

Attention heads

mix information

Classification head

map states to label scores

Their counts are independent.

| Attention mixes states; a task head predicts labels.

A numerical cross-attention example

One query · four source keys · chosen scaled scores

Raghav

goes

school

Delhi

0.2

0.1

0.3

2.0



0.110

0.100

0.122

0.668

softmax across source positions $\rightarrow c_t = \sum_i a_{ti} v_i$

| Chosen scaled scores; probabilities recomputed with softmax.

Computing the weighted sum of values

source	weight α	value vector V
Raghav	0.110	$[1, 0]$
goes	0.100	$[0, 1]$
school	0.122	$[1, 1]$
Delhi	0.668	$[2, 1]$

$$c_t = 0.110[1,0] + 0.100[0,1] + 0.122[1,1] + 0.668[2,1]$$

$$c_t \approx [1.57, 0.89]$$

The weights mix values into $c_t \approx [1.57, 0.89]$.